Abstract

Acknowledgements

Contents

List of Figures

List of Tables

Chapter 1

Strategic Vision and Project Scope

Contents

1.1 General Introduction

Digital transformation has changed how educational institutions manage teaching, communication, assessment, and academic follow-up. Despite this shift, many institutions still rely on fragmented combinations of messaging applications, file-sharing tools, video-conferencing platforms, and manually coordinated assessment processes. Such fragmentation reduces continuity in the learning experience, increases administrative overhead, and weakens the institutional ability to manage educational workflows within one controlled environment.

Eduverse is an integrated educational platform developed to address this problem through a unified, multi-organization, role-based environment. Its main implemented client is a web application, supported by an MVP mobile companion application that extends access to selected academic workflows for students and teachers. The platform is centered on the class as the primary digital workspace in which materials, assignments, examinations, results, notifications, live sessions, academic communication, and intelligent support services can be managed in a coordinated manner. Within this workspace, classes are organized by term and stage, can move to Past Terms when concluded, and preserve academic history under organization-level visibility and feature controls.

At a high level, Eduverse combines a modern web platform with supporting cloud and real-time services in order to provide a realistic and extensible educational system. The web client is built using Next.js, React, TypeScript, Tailwind CSS, and Supabase, while the mobile companion is built using Expo, React Native, and TypeScript. These technologies are mentioned here only as an introductory overview; detailed technical analysis and implementation discussion are reserved for later chapters.

1.2 Problem Statement

Educational institutions often manage closely related academic tasks through separate systems that are not designed to operate as one coherent platform. Materials may be distributed through one service, communication may take place in another, live teaching may depend on a third platform, and assessment records may be maintained elsewhere. As a result, students are forced to move between disconnected interfaces, while teachers and

administrators struggle to maintain consistent oversight of participation, submissions, communication, and evaluation.

This problem becomes more serious in institutions that must manage multiple organizations, classes, and user roles under different access conditions. A practical educational platform must support not only learning activities, but also institutional control, membership management, and role-based access across different organizational contexts. Without these capabilities, digital education systems tend either to become rigid and limited or to become too loosely structured to support reliable academic administration.

A further challenge concerns platform coverage across devices. Rich authoring, administration, and class management workflows are best handled through a full web environment, whereas frequent day-to-day actions such as checking classes, following notifications, viewing assignments, opening materials, joining live sessions, and reading messages must also be accessible on mobile devices. This creates a need for an educational platform centered on a comprehensive web environment, supported by a practical mobile companion rather than a simplistic one-client-for-all approach.

1.3 Project Motivations

The development of Eduverse is motivated by the need to provide a unified academic environment in which essential educational workflows can be carried out without fragmentation. A coherent class-centered system can improve the student experience, reduce operational burden for teachers and administrators, and strengthen institutional visibility over communication, assessment, and academic progress.

The project is also motivated by the growing demand for flexible educational infrastructure. Modern institutions often require platforms that can support multiple organizations, distinct user roles, synchronous and asynchronous learning activities, and selective feature enablement within one extensible environment. Eduverse responds to this need by combining institutional control with practical support for daily teaching and learning workflows.

From an academic perspective, Eduverse is also motivated by its value as a software engineering project. It provides a realistic setting in which requirements analysis, interface planning, system integration, data

management, real-time communication, AI-assisted services, and technical documentation can be studied and implemented within one coherent graduation project context.

1.4 Strategic Objectives

At the strategic level, Eduverse aims to establish an integrated educational platform whose primary implemented client is a web application, complemented by an MVP mobile companion application, in order to centralize essential academic workflows for organizations, teachers, and students within one role-aware and extensible system.

The main strategic objectives of the project are as follows:

- To centralize core educational workflows such as class management, materials, assignments, examinations, results, notifications, communication, and live sessions within one coordinated platform.
- To support multi-organization operation and role-based access for organization administrators, teachers, and students, including memberships spanning multiple organizations or roles with active organization and role switching according to the current workspace context.
- To provide a primary web-based environment capable of supporting both institutional administration and day-to-day academic activity.
- To extend selected high-frequency academic workflows to a mobile MVP companion application without claiming full feature parity with the web platform.
- To incorporate distinctive educational capabilities such as an AI Agent and an integrated coding workspace as part of the platform's broader academic value.
- To organize data and services in a maintainable and scalable manner suitable for educational institutions with different operational needs.
- To produce a graduation project that is both academically meaningful and practically relevant as a modern educational software platform.

1.5 Educational and Professional Goals for the Team

Eduverse also serves clear educational and professional goals for the project team. From an educational standpoint, it provides an opportunity to apply software engineering concepts to a realistic problem domain rather than to an isolated classroom exercise. The team must analyze institutional needs, define system boundaries, organize requirements, structure user roles, and translate these decisions into a functional digital platform.

From a professional standpoint, the project strengthens practical experience in requirements analysis, frontend development, backend integration, database usage, real-time service integration, collaborative development workflows, and technical documentation. It therefore contributes not only to the production of software, but also to the development of disciplined engineering practice, teamwork, and academic reporting skills.

1.6 Project Scope and Boundaries

The scope of Eduverse in this dissertation is defined around the implemented web platform and the existing mobile MVP companion application. Chapter 1 is limited to introducing the problem context, strategic direction, main user groups, system boundaries, and general dissertation structure. Detailed requirements, architecture, database design, implementation decisions, and testing evidence are reserved for later chapters.

1.6.1 System Boundaries and User Roles

Eduverse operates within a multi-organization educational context and therefore distinguishes clearly between administrative, instructional, and learner responsibilities. At the institutional level, the platform supports organization administrators who manage organizations, user access, public and invitation-based membership onboarding, public join links, approval-based join requests, feature configuration, class oversight, teacher permission controls, and operational control. A user may belong to multiple organizations, and the same organization member may hold more than one active role, with the interface and permissions resolved according to

the currently selected organization and role. At the instructional level, teachers manage class activities such as materials, assignments, live sessions, examinations, and academic follow-up within the permissions defined by organization settings. At the learner level, students access the classes available to them in order to study materials, follow announcements, submit work, join sessions, communicate, and review academic outcomes.

These role distinctions define important system boundaries. Eduverse is not a generic social platform in which all users have the same functional position. Instead, access is governed by organizational membership, class context, and academic responsibility. This role-aware structure supports accountability, controlled visibility, and more reliable institutional management.

1.6.2 Functional Scope

Within its current implemented scope, Eduverse includes authentication, organization access and multi-organization membership, class management, materials, assignments, exams, results and academic history, notifications, live sessions, an AI Agent, and an integrated coding workspace. Together, these functions define the platform as more than a basic learning content portal. Eduverse is intended to support both academic management and active instructional workflows within one system. Within this scope, classes can be visible across the organization, require term and stage definition, and move into Past Terms without losing preserved academic history. Teacher management abilities, class feature controls, student linkage to previous terms, and Results visibility are also governed through organization and class settings.

At the same time, Chapter 1 remains introductory in nature. It does not attempt to document detailed database structures, API behavior, or low-level internal logic. Those aspects are part of the later analytical and technical chapters and lie outside the present scope.

1.6.3 Web and Mobile Client Scope

The main implemented Eduverse client is the web application. It provides the widest operational coverage of the platform, including organization-aware navigation, role-based access, membership handling, class management, materials, assignments, exams, results, notifications, live sessions, AI Agent

workflows, and the integrated coding workspace. For this reason, the web application should be regarded as the core working environment of the system.

The mobile application has a more focused scope. It is an MVP companion application intended to extend access to selected academic workflows such as authentication, dashboard viewing, classes, assignments, notifications, chat, materials, and live sessions. Although it is connected to the same broader platform environment, it is not yet feature-complete relative to the web application and should therefore be described as a companion application rather than a replacement for the primary platform.

1.6.4 Competitive Features and Technological Innovation

Eduverse is distinguished by the combination of features it brings together in one educational platform. A major distinguishing point is its multi-organization, role-based structure, which allows the same system to support organizational control, instructional management, and student participation in a coordinated way. The platform also integrates live sessions, an AI Agent, and a coding workspace within the same broader academic environment rather than treating them as isolated external utilities. It also supports controlled public and invitation-based class access, term-and-stage class organization, and the transition of completed classes into Past Terms while preserving historical records.

Another important feature is the platform's attention to examinations and assessment integrity. Eduverse does not treat evaluation as a peripheral task, but as an integral part of the educational workflow that must be managed within a controlled digital environment, including controlled student-facing Results visibility where needed. In addition, the system deliberately combines a comprehensive web platform with an MVP mobile companion application, thereby balancing rich institutional workflows with accessible everyday mobile use.

From a technological perspective, Eduverse also reflects a deliberate service separation strategy. Structured and frequently queried academic data are stored through Supabase PostgreSQL, large educational files are handled through AWS S3, and real-time live-session interaction is supported through LiveKit. This separated storage and service model strengthens the

platform's practicality for an educational environment that must manage both relational records and larger real-time or file-based workloads.

1.7 Report Organization

This dissertation is organized as follows. Chapter 2 presents the literature review and theoretical framework. Chapter 3 describes the adopted methodology. Chapter 4 covers system analysis and requirements. Chapter 5 presents system design. Chapter 6 discusses software and engineering implementation. Chapter 7 presents testing and evaluation. Chapter 8 addresses future work and planned extensions, followed by the conclusion and appendices.

Chapter 2

Literature Review and Theoretical Framework

Contents

2.1 Introduction

This chapter presents the theoretical background for Eduverse. It reviews the evolution of digital learning systems, the limitations of fragmented educational tools, selected concepts from artificial intelligence in education, and the main foundations that inform the system's design. The discussion draws selectively from the literature on Learning Management Systems, Artificial Intelligence in Education, Role-Based Access Control, and educational platform architecture.

2.2 Evolution of Digital Education Platforms

2.2.1 From Traditional Classroom Management to Digital Learning Systems

Educational management has historically depended on face-to-face coordination, manual attendance records, printed materials, and locally maintained assessment processes. As educational institutions expanded in scale and digital communication became more common, Learning Management Systems emerged to centralize course materials, announcements, assignment handling, and academic tracking within a unified digital environment [1, 2].

The move toward digital learning systems did not simply replace paper-based administration with electronic storage. It introduced new expectations about continuous access, asynchronous participation, online communication, and traceable academic workflows. Modern educational platforms are therefore judged not only by their capacity to store materials, but also by their ability to support coordination, interaction, assessment, and institutional oversight across different users and contexts [11].

2.2.2 Limitations of Fragmented Learning Management Tools

Although digital tools have become common in education, many institutions still depend on separate services for class management, communication, file

sharing, video sessions, assessment, and student support. This fragmented model creates operational discontinuity because activities that belong to the same academic process are distributed across unrelated systems. As a result, students must move repeatedly between platforms, while teachers and administrators face a reduced ability to maintain clear academic oversight [11, 2].

When communication, submissions, live sessions, and evaluation records are distributed across multiple tools, consistency, accountability, and access governance become harder to maintain. The limitation is therefore organizational as well as technical, because it weakens the integrity of the educational environment itself.

2.3 Research Gap and Proposed Integrated Learning Ecosystem

A clear gap exists between the broad range of activities required in modern education and the way those activities are often supported by disconnected systems. Institutions may rely on one tool for class coordination, another for messaging, another for live teaching, another for assignments, and still others for assessment, AI support, or programming practice. While each tool may be effective in isolation, the overall experience remains fragmented and operationally inefficient.

Eduverse addresses this gap by proposing an integrated learning ecosystem rather than a collection of separate utilities. It brings class management, communication, materials, assessment, live sessions, AI-assisted support, and programming practice into one organizationally governed environment. The main implemented client is the web application, while the mobile client remains an MVP companion for selected daily-use workflows. This balance between integration and controlled scope is central to the academic identity of the project. It also reflects the need for one platform to coordinate controlled public access, role-aware governance, and preserved academic history across active and archived class terms.

2.4 Artificial Intelligence in Education

Artificial intelligence in education is commonly discussed as a means of augmenting teaching and learning rather than replacing teachers or institutional judgment [8, 6]. In educational settings, AI systems are valuable when they improve access to explanations, support learning continuity, reduce routine workload, and provide structured assistance without undermining academic responsibility. In Eduverse, this perspective frames the AI Agent as a bounded academic support service embedded in the platform rather than as an authority over teaching or evaluation.

2.4.1 Context-Aware Learning Assistance

One of the most important ideas in AI-supported education is that assistance is more useful when it is tied to the learner's actual academic context. Generic responses are often less effective than support that takes into account course materials, class discussion, ongoing tasks, and instructional expectations. Context-aware assistance can therefore improve relevance, reduce confusion, and support more focused academic interaction [6].

In Eduverse, this means the AI Agent operates within the active class, user role, and immediate learning task so that support remains relevant to the current course activity. Its role is to assist understanding and clarification rather than to substitute for teaching.

2.4.2 Material Summarization and Learning Support

Material summarization represents a practical form of AI assistance in education. Long documents, learning materials, and instructional resources can be difficult for students to process efficiently, especially when they are working under time constraints or trying to review several academic topics in parallel. AI-assisted summarization can support comprehension by highlighting main points, reducing information overload, and helping learners return more quickly to the central ideas of a topic [8].

This kind of support is most appropriate when it complements reading rather than replacing it. In Eduverse, it is most relevant to class materials and related course resources, where AI-generated summaries can complement class-based AI interaction while keeping the original material central.

2.4.3 Assessment and Feedback Support

AI can also support assessment-related workflows by assisting with draft guidance, formative feedback, and structured academic support around tasks and examinations. In theory, such support can help learners prepare, reflect on their work, and identify areas that require improvement [6]. However, educational literature also emphasizes that AI use in assessment must be governed carefully so that it supports learning without weakening academic integrity or teacher responsibility [8, 6].

For Eduverse, this means the AI Agent may assist students and teachers with preparatory guidance, clarification, and formative support around assignments and examinations, while final instructional judgment and assessment responsibility remain with the teacher. During active assessed work, such support must remain bounded and should not provide direct final answers in place of genuine student effort. This governance also extends to controlled student-facing Results visibility as part of assessment integrity.

2.5 Benchmarking: Fragmented Tools vs. Integrated Eduverse Approach

Table 2.1 summarizes the difference between a fragmented educational-tool model and the integrated approach proposed by Eduverse.

2.6 Theoretical and Technical Foundations of Eduverse

2.6.1 Web-Based Educational Platforms

Web-based educational platforms remain the strongest medium for full academic management because they support richer interfaces, broader administrative workflows, and deeper integration across organizational functions. They are especially suitable for tasks such as class configuration, materials management, assessment setup, and detailed instructional coordination.

From a theoretical standpoint, web environments are architecturally well suited to full academic management because they can centralize complex

Table 2.1. Fragmented Educational Tools vs. Integrated Eduverse Approach

Educational Need	**Fragmented Tool Approach**	**Integrated Eduverse Approach**
Class management	Class records, memberships, and academic coordination are handled through separate administrative tools or partially manual processes.	Class organization is managed within one platform through organization-aware and role-based structures.
Communication	Messaging and announcements are distributed across external chat or notification tools.	Communication remains connected to the academic environment and class context.
Live sessions	Synchronous teaching depends on external meeting services disconnected from course workflows.	Live sessions are treated as part of the broader educational platform and linked to class participation.
Materials and assignments	Learning resources and tasks are often stored, shared, and submitted through different services.	Materials and assignments are coordinated within the same academic workspace.
AI support	AI tools are usually external and not directly tied to the learner's course context.	AI support is positioned within the class environment as contextual academic assistance.
Programming practice	Coding activities, when needed, often require separate platforms or disconnected lab tools.	Programming practice can be supported through an integrated coding workspace within the platform ecosystem.
Assessment	Examinations and grading may rely on detached tools with limited continuity or oversight.	Assessment workflows are integrated into the same role-aware educational system.

interfaces, persistent records, administrative controls, and cross-workflow coordination within one accessible system.

2.6.2 Mobile Companion Learning Applications

Mobile educational applications play a different but still important role in digital learning environments. Their strength lies in quick access, timely participation, and support for routine high-frequency actions such as checking notifications, reviewing assignments, opening materials, and joining live sessions. As a result, they are especially useful as companion systems for daily academic engagement.

In Eduverse, the mobile client should therefore be understood as an MVP companion rather than as a complete replacement for the main platform. It extends accessibility to core user-facing workflows while more comprehensive management and authoring remain centered in the web application.

2.6.3 Real-Time Virtual Classrooms

Synchronous learning environments are an important part of modern educational theory because they strengthen immediacy, interaction, and instructional presence. When live classrooms are integrated into the broader academic platform rather than isolated in external services, continuity between teaching, communication, and follow-up becomes easier to maintain [5].

This foundation is relevant to Eduverse because live sessions are not treated as a separate product layer. Instead, they are part of the integrated learning environment and serve the broader goal of linking synchronous participation with the rest of the class workflow, including session chat, presentation annotation, and controlled whiteboard participation.

2.6.4 Integrated Coding Environments for Programming Education

Programming education often requires an environment in which learners can write code, inspect results, practice problem solving, and remain connected to the instructional context of the class. When coding practice is separated entirely from the educational platform, students may lose continuity between learning materials, teacher guidance, assignments, and practical execution.

An integrated coding environment addresses this problem by placing programming practice within the same educational ecosystem as the rest of the academic workflow. For Eduverse, this direction is interpreted pragmatically as a browser-safe coding workspace with guided runners and extension-launch support rather than as a full backend IDE or unrestricted backend execution environment.

2.6.5 Security, Roles, and Data Management

Trustworthy educational platforms depend on controlled access, clear role distinctions, and dependable data management. Role-Based Access Control is especially relevant in this context because it supports structured separation between administrative, teaching, and learner permissions [10, 4]. In educational systems, such distinctions are necessary not only for security, but also for accountability and appropriate institutional governance. In Eduverse, this perspective also supports multi-role memberships in which access is resolved according to the active role within the current organization. In broader institutional terms, this kind of governance also aligns with active-role selection, controlled public access requests, and permission-scoped teacher management.

Data management is equally foundational. Educational systems must maintain reliable records, protect assessment-related information, and support different types of content ranging from structured academic data to larger learning materials and live-session interaction. In Eduverse, this also supports separation between relational academic records and larger instructional files so that storage handling, permissions, and delivery can be controlled more appropriately. It also supports the preservation of historical class records across archived terms so that academic continuity does not depend only on the active class state. From a theoretical perspective, this requires a design orientation that treats security, data organization, and permissions as core educational infrastructure rather than as secondary technical concerns. This view is consistent with Eduverse, where access control, data separation, and institutional structure are part of the platform's trust model.

Taken together, these theoretical threads show that effective educational platforms must reduce fragmentation while coordinating complementary forms of support across web, mobile, synchronous, and practice-oriented learning contexts. They also indicate that AI assistance, real-time

instruction, integrated coding environments, and secure data management are most valuable when they operate within one coherent academic system rather than as detached services. For Eduverse, this combined perspective provides the conceptual basis for a unified platform model with differentiated client roles, bounded AI support, live-session continuity, and protected institutional workflows. The following chapters build on this foundation to explain the analysis, design, and implementation choices of the project.

Chapter 3

Methodology

Contents

3.1 Introduction

Choosing a development methodology for Eduverse was not a routine decision. The platform's modules, including organization governance, class operation, learning content, assessment, communication, AI support, and integrated IDE support, are not independent; each depends on shared permissions, shared data, and the roles of the Organization Administrator, Teacher, and Student. A methodology that fixed requirements and structure at the outset, before these dependencies were understood, risked producing modules that worked in isolation but conflicted once integrated.

This chapter presents the methodological reasoning behind Eduverse's development: what approach was chosen, why it suited a multi-organization educational platform, and how it was applied consistently across features. Engineering practices and tool-specific execution are addressed separately in Chapter 6.

3.2 System Development Methodology

Eduverse was developed using a hybrid iterative methodology with predominantly feature-based vertical-slice development, rather than a strict waterfall model in which analysis, design, implementation, and validation each occur once in a fixed sequence. The platform instead evolved through repeated increments, each centered on a meaningful academic or administrative capability, allowing requirements and structure to be refined as the relationships between modules became clearer through development.

3.2.1 Feature-Based Iterative Methodology

Development was organized around functional areas rather than technical layers. Iterations typically focused on modules such as organization management, class management, materials, assignments, examinations, live sessions, results, AI Agent support, and integrated IDE support. Within each iteration, the selected feature moved through recurring stages of analysis, design, development, and validation, with the outcome of one increment informing the next.

This structure also governed coordination between the web application, which served as the primary client, and the MVP mobile companion

application. Because the mobile app targeted selected high-frequency academic actions rather than full feature parity, each iteration served as a checkpoint for deciding which workflows required cross-client refinement and which remained web-centered.

3.2.2 Vertical-Slice Development Approach

A feature was generally treated as a coherent functional unit rather than split into isolated interface-only or backend-only stages. Depending on its scope, a slice could span user-interface behavior, business rules, backend or API behavior, permission handling, and data requirements. This suited a role-based platform: an action such as submitting an assignment or scheduling an examination cannot be understood as a user-interface concern alone, since it also depends on who may perform it, what rules govern it, and how the resulting record is stored and retrieved.

This description is deliberately framed as predominantly vertical-slice rather than absolute. Some features required only a subset of system layers, while others extended across several and, in selected cases, into the MVP mobile companion application as well.

3.2.3 Justification for Methodology Selection

A rigid sequential methodology was a poor fit for Eduverse because the platform's module boundaries and dependencies were not fully knowable in advance; they became clear through implementation. Organization governance, class operation, learning content, assessment, communication, AI support, and integrated IDE support influence one another through shared permissions, shared data, and shared user context, so fixing their design before that interplay was understood would have risked structural rework later.

The iterative, vertical-slice approach let the project refine permissions, workflows, interface behavior, and data structures incrementally, checking after each increment whether a feature remained coherent with the wider platform before expanding further. This mattered in particular because the Organization Administrator, Teacher, and Student each interact with related features from different perspectives, so role-based access and workflow consistency needed repeated, not one-time, validation.

3.3 Iterative Development Structure

Although development was iterative, each increment followed the same underlying cycle, adjusted in scope and depth to the complexity of the feature involved. Some increments centered mainly on the web application; others required supporting changes to the backend, database, permissions, or MVP mobile companion application. What stayed constant was not the shape of each iteration but the sequence it followed, moving from clarification, through realization and validation, to integration.

Table 3.1. Repeated Iterative Development Structure in Eduverse

Repeated phase	**Methodological focus**
Feature selection and scope clarification	Identify the next meaningful academic or administrative feature and define the intended workflow boundaries for the current increment.
Workflow and requirement analysis	Examine the target actors, conditions, information flow, constraints, and dependencies associated with the selected feature.
Design of feature behavior and user interaction	Define how the feature should behave from the user perspective, including role-sensitive interaction, states, and decision points.
Feature-slice implementation	Realize the selected feature across the relevant system layers, including interface behavior, services, permissions, and data handling where the scope requires them.
Validation and refinement	Review the implemented behavior against the intended workflow, resolve inconsistencies, and refine the feature before broader adoption.
Integration into the wider platform	Connect the completed feature to the surrounding platform so it remains coherent with existing modules, shared rules, and user workflows.

3.4 Chapter Summary

Eduverse followed an iterative, feature-based methodology with predominantly vertical-slice development. Through this structure, the project was developed in repeated increments centered on meaningful academic and administrative features, with each increment progressing through analysis, design, implementation, validation, and integration. The next chapters move from methodology to system analysis, system design, implementation, and testing, where the platform's structure and technical realization are presented in greater detail.

Chapter 4

System Analysis and Requirements

Contents

4.1 Introduction

This chapter presents the system analysis and requirements of Eduverse. It defines the scope of the platform from an analytical perspective, identifies the principal actors, organizes the functional and non-functional requirements, and documents the updated use-case and activity models that describe the main user-visible workflows.

The analysis is grounded in the current Eduverse platform scope represented by the updated use-case diagrams and supporting project documentation. The chapter therefore focuses on verified platform behavior, configurable feature availability, and controlled role-based interaction rather than on low-level implementation detail.

4.2 System Overview

Eduverse is a multi-organization educational platform centered on the class as the main operational workspace. Its primary implemented client is the web application, while the companion mobile scope supports selected high-frequency academic actions. Within this environment, organizations can manage users, roles, class structures, course materials, communication, assignments, examinations, live sessions, results, profile settings, and help resources through one coordinated system.

The platform is role-aware and configuration-sensitive. A single account may belong to more than one organization and may hold more than one role, so the visible pages and permitted actions depend on the selected Active Organization, the Active Role, organization-level settings, class-level feature settings, and class membership. As a result, Eduverse must be analyzed not only as a collection of learning tools, but also as a governed workspace in which institutional control and daily educational activity are resolved together.

4.3 System Actors

The main human actors of Eduverse are the Student, the Teacher, and the Organization Administrator. These actors do not represent separate accounts by necessity; the same user may hold more than one role, may belong to more

than one Organization, and may switch context according to the selected workspace.

Table 4.1. Primary Eduverse System Actors

Actor	**Analytical Role in the System**
Organization Administrator	Manages organization setup, member access, public access settings, teacher permissions, organization defaults, class oversight, and organization-level dashboards.
Teacher	Manages instructional workflows such as class operation, materials, communication, assignments, examinations, live sessions, results follow-up, AI-supported tasks, and coding-lab activities within granted permissions.
Student	Participates in classes by studying materials, using communication tools, submitting assignments, taking examinations, joining live sessions, using available AI support, and reviewing released results.

In practice, the actor visible to the interface is determined by the current workspace context. The selected organization, the Active Role, enabled organization features, enabled class features, and actual class membership all influence what the user can view or modify at a given moment.

4.4 Functional Requirements

The functional requirements of Eduverse are grouped by module so that the system scope remains aligned with the updated use-case model and the current platform terminology.

Authentication and Access

- **FR-01.** The system shall allow users to sign up, sign in, verify email addresses, request password reset, and log out.

- **FR-02.** The system shall authenticate access before opening organization-scoped or class-scoped workspace routes.
- **FR-03.** The system shall allow users with more than one eligible workspace context to select the Active Organization and Active Role.
- **FR-04.** After successful authentication and context resolution, the system shall open the role-appropriate workspace or dashboard.

Organization and Role Management

- **FR-05.** The system shall allow an Organization Administrator to create an Organization and configure its default settings.
- **FR-06.** The system shall allow administrators to invite, register, activate, and manage organization members and their roles.
- **FR-07.** The system shall support invitation-based access and, when enabled, Public Join Link access with target-role selection and direct-join or approval-based entry modes.
- **FR-08.** The system shall allow administrators to review join requests, control public organization settings, manage teacher class permissions, and associate previous-term student history where applicable.

Class Management and Class History

- **FR-09.** The system shall allow authorized users to create and edit classes with details such as title, code, term, stage, description, and related settings.
- **FR-10.** The system shall allow class membership management, including assigning or changing the responsible Teacher and managing student enrollment according to permissions.
- **FR-11.** The system shall allow class-level configuration of enabled tools, visibility rules, and Results visibility when these controls are available.

- **FR-12.** The system shall allow administrators or permitted teachers to end a class, stage, or term and move the class into Past Terms while preserving class history.

Course Content and Materials

- **FR-13.** The system shall allow authorized users to upload materials together with descriptive details and the selected file.
- **FR-14.** The system shall allow class members to view, search, filter, preview, open, and download available materials according to access permissions.
- **FR-15.** The system shall allow authorized deletion of materials and, when AI support is enabled, provide material-related study summaries or AI context support.

Class Communication and Notifications

- **FR-16.** The system shall allow class members to send text or media messages within the class communication space.
- **FR-17.** The system shall allow authorized users to post announcements, remove announcement entries from the announcement area, and search the class conversation.
- **FR-18.** The system shall generate notifications from relevant system events and shall allow users to view, open, mark, mark all, and delete notification entries.

Assignment Management

- **FR-19.** The system shall allow authorized users to create, edit, delete, save as draft, and publish assignments.
- **FR-20.** The system shall allow assignment configuration for due date, score range, submission modes, and optional attached files.

- **FR-21.** The system shall allow students to submit text, file, or combined work according to assignment settings and to resubmit when the assignment policy permits it.
- **FR-22.** The system shall allow teachers and administrators to review submissions, record grades, provide feedback, and use AI-supported feedback assistance where enabled.

Exam Management and Integrity

- **FR-23.** The system shall allow authorized users to create, edit, delete, draft, and publish exams, including question management and passcode configuration when required.
- **FR-24.** The system shall allow eligible students to start an exam attempt, enter a passcode when required, answer questions, and submit the attempt within the valid exam window.
- **FR-25.** The system shall apply configured integrity controls, including fullscreen requirements and integrity-event recording when the required exam state is interrupted.
- **FR-26.** The system shall allow teachers and administrators to monitor attempts, review integrity events, grade manual answers, void attempts, and grant retakes where permitted.
- **FR-27.** The system shall allow release of exam results and, after release, shall allow students to view result details and answer review where the class settings permit it.

Live Sessions and Whiteboard

- **FR-28.** The system shall allow authorized class managers to start, manage, and end live sessions.
- **FR-29.** The system shall allow eligible participants to join a live session and use session tools such as microphone, camera, screen sharing, participants view, and session chat.

- **FR-30.** The system shall allow whiteboard interaction or whiteboard viewing according to the role and permissions of the current participant.

Results and Dashboards

- **FR-31.** The system shall provide role-specific dashboards for Students, Teachers, and Organization Administrators.
- **FR-32.** The system shall allow students to view assignment grades, feedback, released exam results, and related result details within permitted classes.
- **FR-33.** The system shall allow teachers and administrators to review class results, assignment results, and exam results subject to the configured visibility rules.

AI Learning Support

- **FR-34.** The system shall provide a class AI Agent for contextual academic support when the AI feature is enabled.
- **FR-35.** The system shall allow AI-assisted study support such as material summaries and assignment-related guidance without replacing the original material or teacher judgment.
- **FR-36.** The system shall allow AI-assisted drafting of assignments, exams, and feedback only where the corresponding AI tools are enabled and available.

Integrated Coding Lab / IDE Extension

- **FR-37.** The system shall allow users to open the Integrated Coding Lab / IDE Extension when it is enabled for the current class.
- **FR-38.** The system shall allow users to choose an available coding environment and manage the visible project files in the workspace.
- **FR-39.** The system shall allow users to open, edit, save, and run code, and to preview supported web or markdown output.

- **FR-40.** The system shall allow users to review console or problems output, reset the workspace when needed, and use the IDE terminal where that capability is supported.

Profile and Help

- **FR-41.** The system shall allow users to view profile information, change theme preferences, and manage password-related account settings.
- **FR-42.** The system shall allow users to switch the Active Organization and Active Role through the available workspace controls.
- **FR-43.** The system shall provide searchable help content for account, class, administration, AI, coding, assignment, exam, results, navigation, and troubleshooting topics.

4.5 Non-Functional Requirements

The main non-functional requirements of Eduverse are summarized below.

- **NFR-01 Usability.** The interface shall keep common academic workflows understandable for students, teachers, and administrators, with clear navigation between organization, class, and feature contexts.
- **NFR-02 Security and access control.** Access to organizations, classes, files, assignments, examinations, results, and live-session resources shall be protected through authenticated and role-aware control.
- **NFR-03 Reliability.** The system shall preserve correct state for memberships, materials, submissions, exam attempts, notifications, and released results during regular operation.
- **NFR-04 Performance.** The system shall provide acceptable responsiveness for dashboards, materials, assignments, results, chat, and live-session interaction under normal academic usage.

- **NFR-05 Maintainability.** The platform shall remain modular enough to support continued refinement of class tools, AI features, assessment workflows, and coding-lab capabilities.
- **NFR-06 Scalability.** The system shall remain suitable for multiple organizations, growing class counts, increasing user membership, and larger volumes of academic content and records.
- **NFR-07 Data privacy.** Personal and academic data shall remain visible only to permitted users, especially for grades, submissions, examinations, notifications, and organization-scoped records.
- **NFR-08 Configurable feature availability.** The system shall respect organization-level and class-level feature configuration so that disabled tools are hidden or blocked in a controlled and understandable manner rather than failing unpredictably.

4.6 Use-Case Model

The use-case model describes Eduverse through the interactions between the primary human actors and the services available to them within the selected organization and class context. The Project Overview diagram is a summary of the complete system, while the remaining diagrams focus on specific modules of behavior.

Project Overview

Figure 4.1 presents the high-level summary of Eduverse and shows how the Student, Teacher, and Organization Administrator relate to the platform's principal service areas.

Authentication and Access

Figure 4.2 represents the entry and workspace-access flow. It includes registration, sign-in, email verification, password reset, logout, and the conditional selection of the Active Organization and Active Role when more than one eligible context exists.

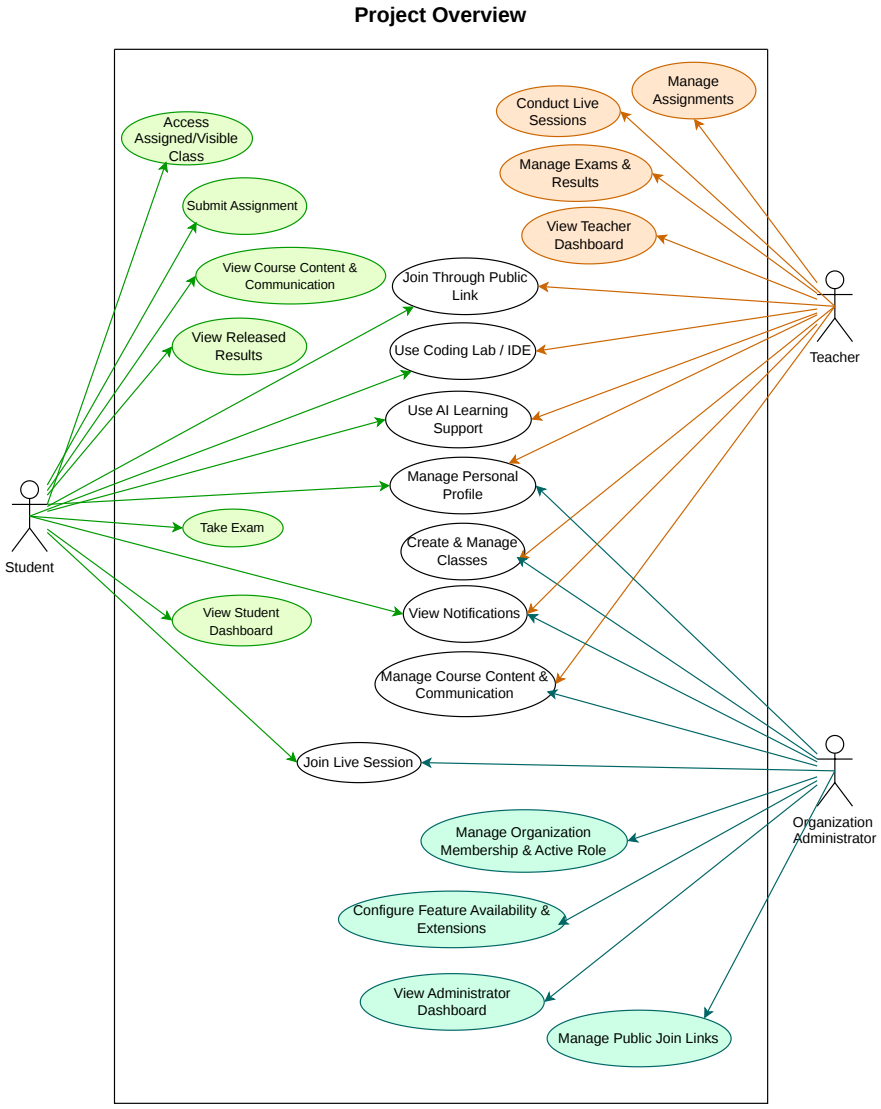

Figure 4.1. Project Overview Use-Case Diagram

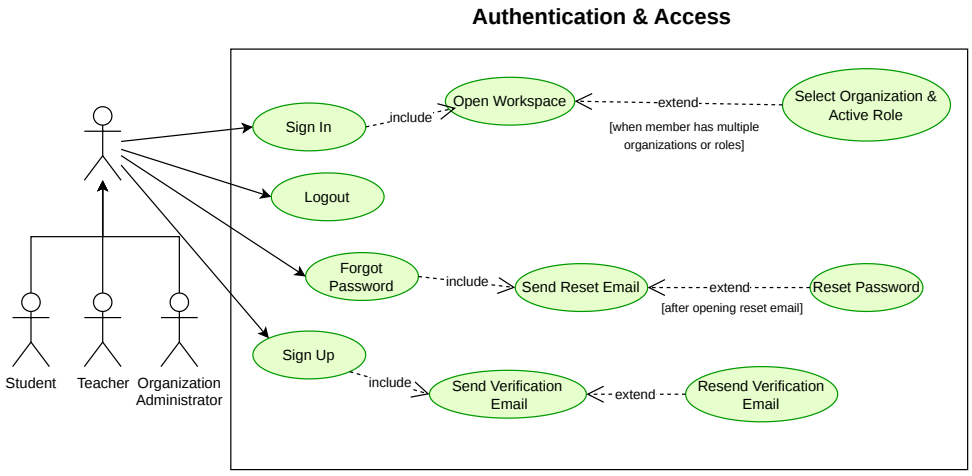

Figure 4.2. Authentication and Access Use-Case Diagram

Organization and Role Management

Figure 4.3 models organization-scoped governance. It covers organization creation, member invitation or registration, public join access, join requests, role selection, active-context switching, public organization settings, teacher class permissions, and previous-term linkage where applicable.

Class Management

Figure 4.4 positions the Class as the core operational unit of the platform. The diagram emphasizes class creation and editing, member and teacher management, feature configuration, visibility control, class ending, and movement to Past Terms while preserving history.

Assignment Management

Figure 4.5 represents the assignment workflow from authoring and publication to submission, grading, and feedback. Its conditional relations are especially meaningful for submission-mode choices and draft-versus-published assignment state.

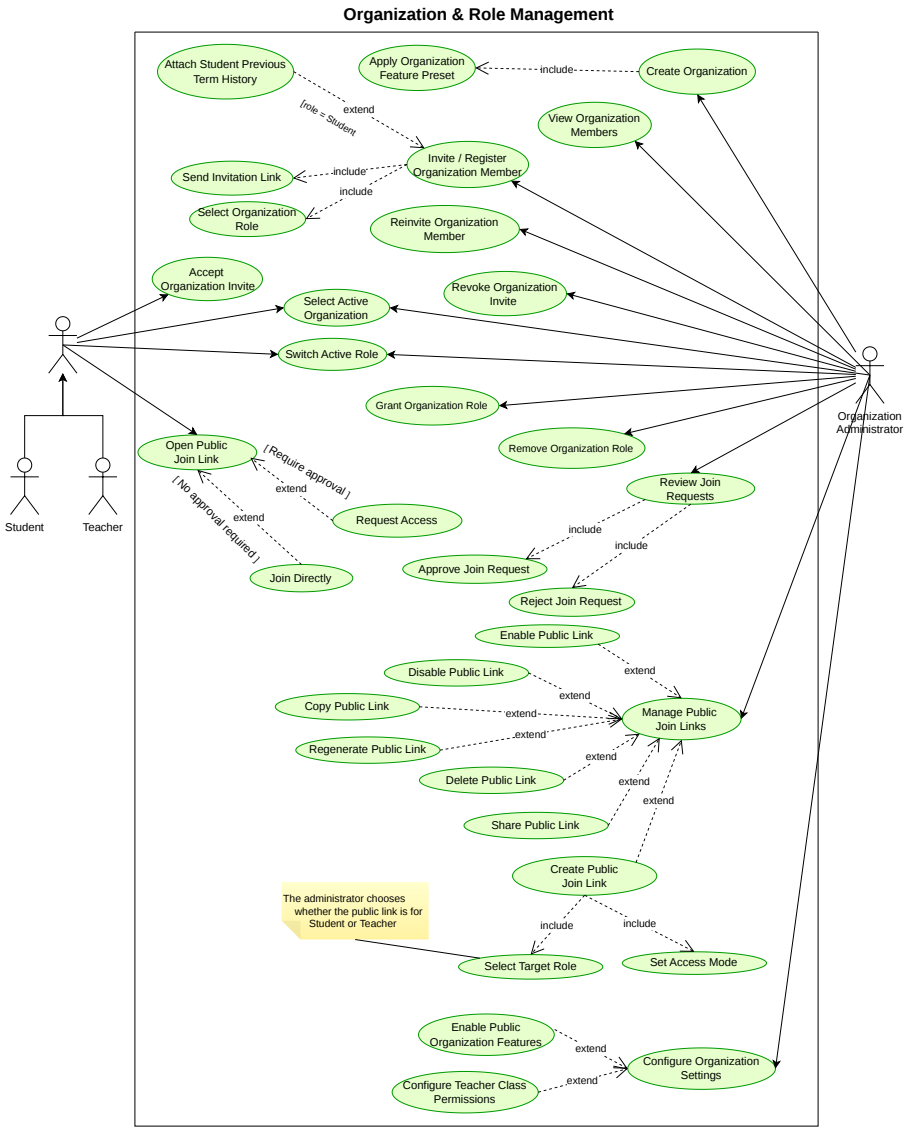

Figure 4.3. Organization and Role Management Use-Case Diagram

Figure 4.4. Class Management Use-Case Diagram

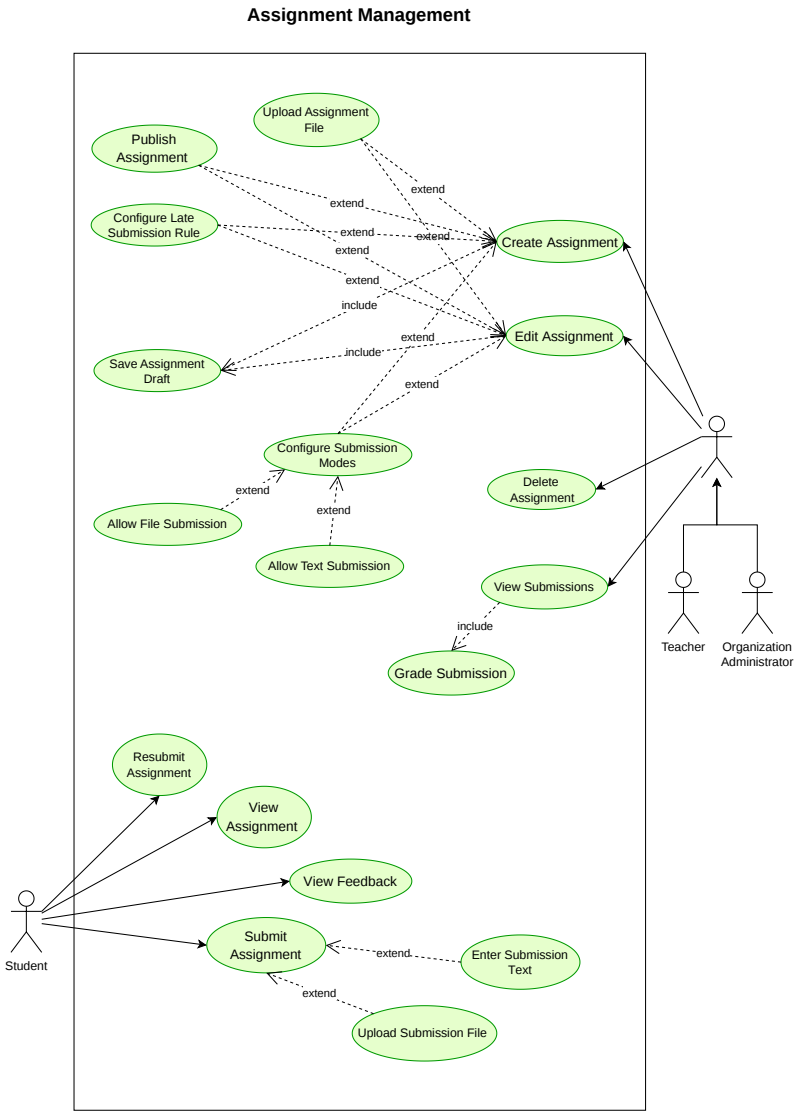

Figure 4.5. Assignment Management Use-Case Diagram

AI Learning Support

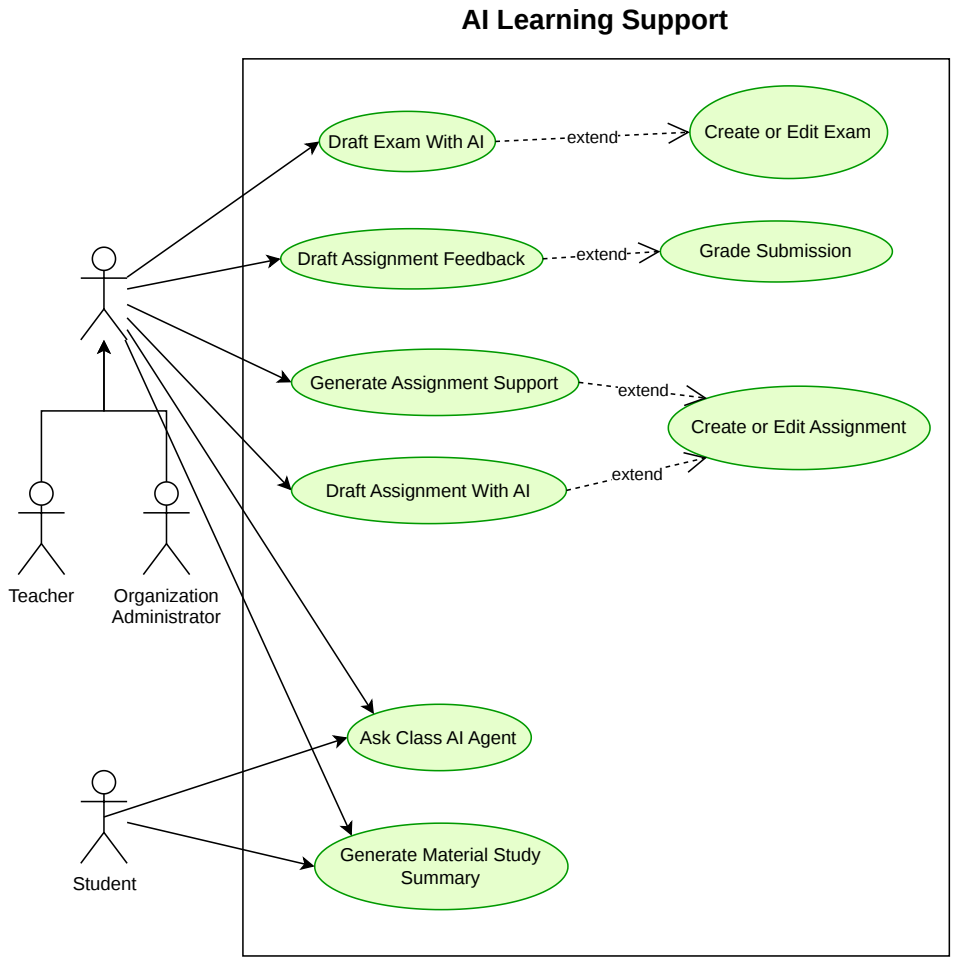

Figure 4.6. AI Learning Support Use-Case Diagram

Figure 4.6 shows how the AI layer acts as contextual academic support rather than as a primary actor. The diagram captures the class AI Agent, material summaries, assignment guidance, and AI-supported drafting tasks that may be available when the feature is enabled.

Class Communication and Notifications

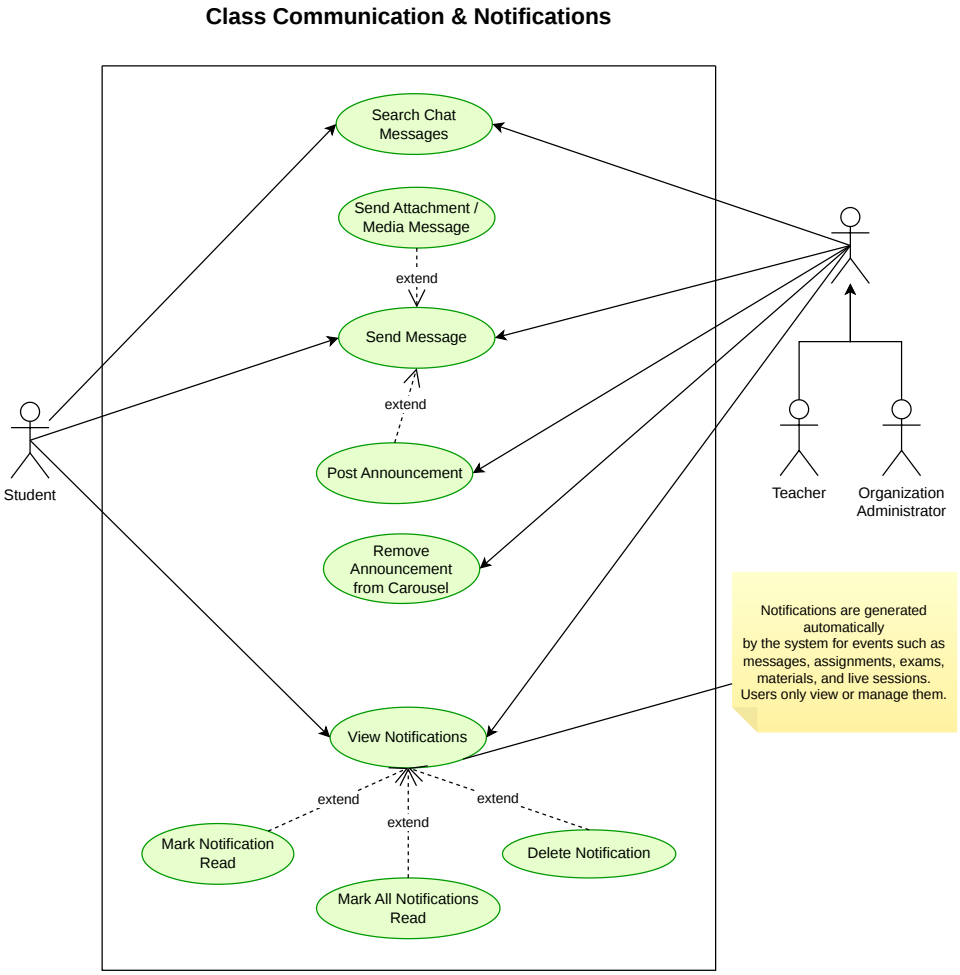

Figure 4.7. Class Communication and Notifications Use-Case Diagram

Figure 4.7 distinguishes between direct class interaction and event-driven awareness. Messages, announcements, and chat search are user-initiated, while notifications are system-generated and are managed only through viewing, read-state handling, and deletion actions.

Course Content and Materials

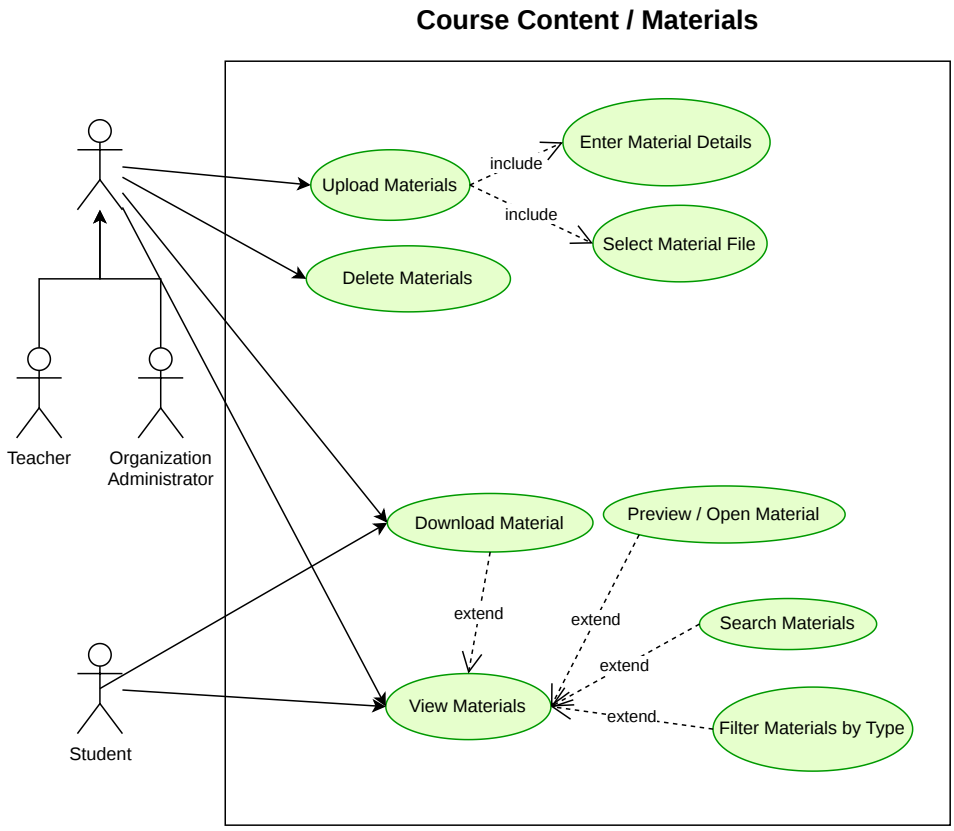

Figure 4.8. Course Content and Materials Use-Case Diagram

Figure 4.8 models the material life cycle inside the class context. Uploading, material details, file selection, viewing, search, filtering, preview, download, and deletion are all organized around controlled class access.

Exam Management and Integrity

Figure 4.9 describes the controlled assessment workflow. Important conditional relations include passcode entry when required, integrity

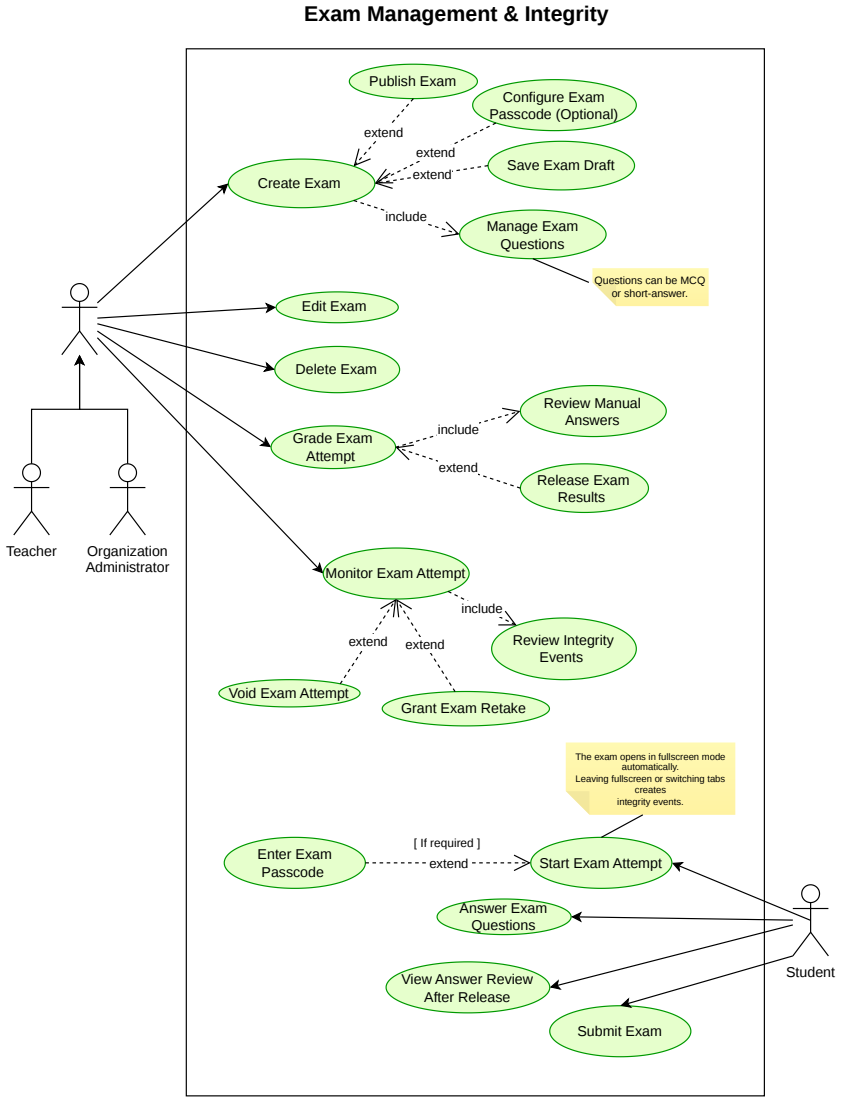

Figure 4.9. Exam Management and Integrity Use-Case Diagram

monitoring during the attempt, review of integrity events, result release, retake handling, and answer review after release.

Live Sessions and Whiteboard

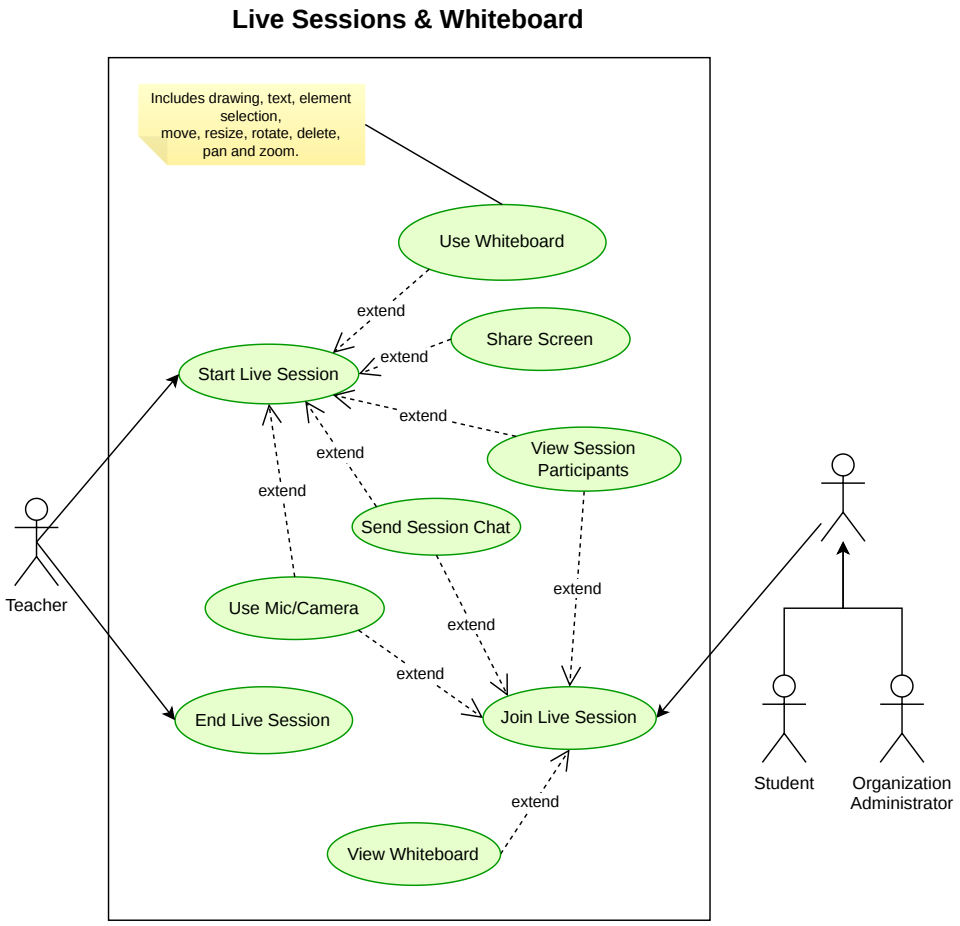

Figure 4.10. Live Sessions and Whiteboard Use-Case Diagram

Figure 4.10 captures the synchronous class environment. It centers on starting, joining, and ending sessions, while microphone, camera, screen

sharing, participants, session chat, and whiteboard use appear as key collaboration capabilities.

Integrated Coding Lab / IDE Extension

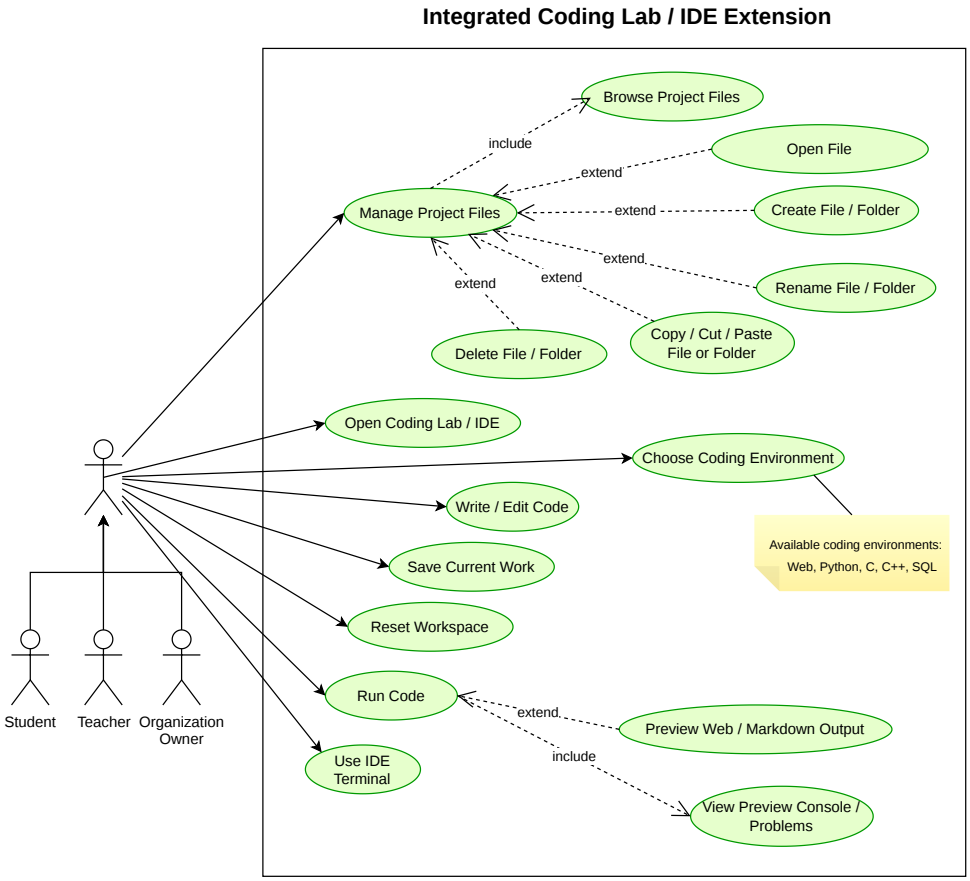

Figure 4.11. Integrated Coding Lab / IDE Extension Use-Case Diagram

Figure 4.11 models the programming workspace integrated into Eduverse. It includes environment selection, file management, editing, saving, running code, previewing output, inspecting console or problem views, resetting the workspace, and using the IDE terminal where supported.

Profile and Help

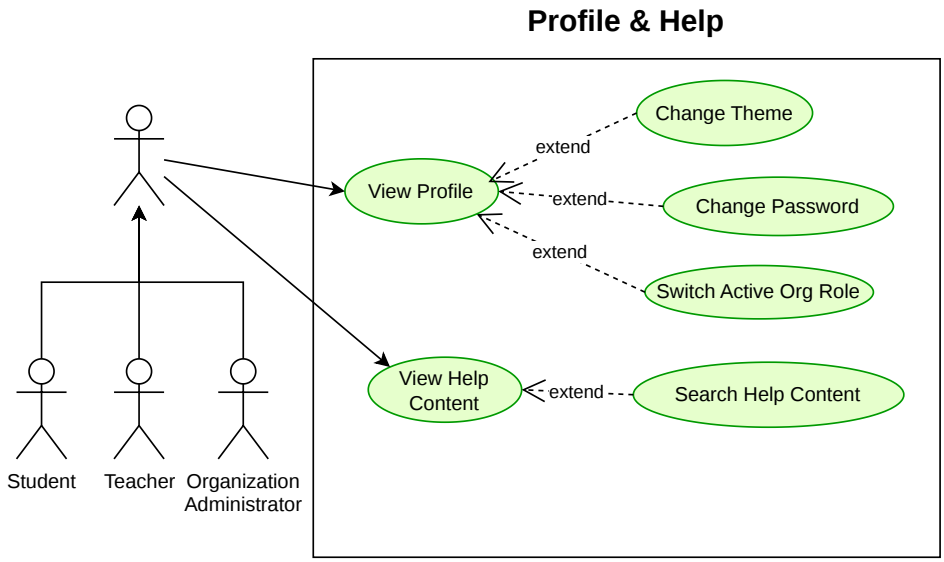

Figure 4.12. Profile and Help Use-Case Diagram

Figure 4.12 covers profile-oriented and support-oriented functions such as viewing the profile, changing password or theme settings, switching role context, opening help resources, and searching help content.

Results and Dashboards

Figure 4.13 emphasizes differentiated results visibility across actors. It combines role-specific dashboards with class results, assignment results, exam results, released student outcomes, feedback, and answer review where permitted.

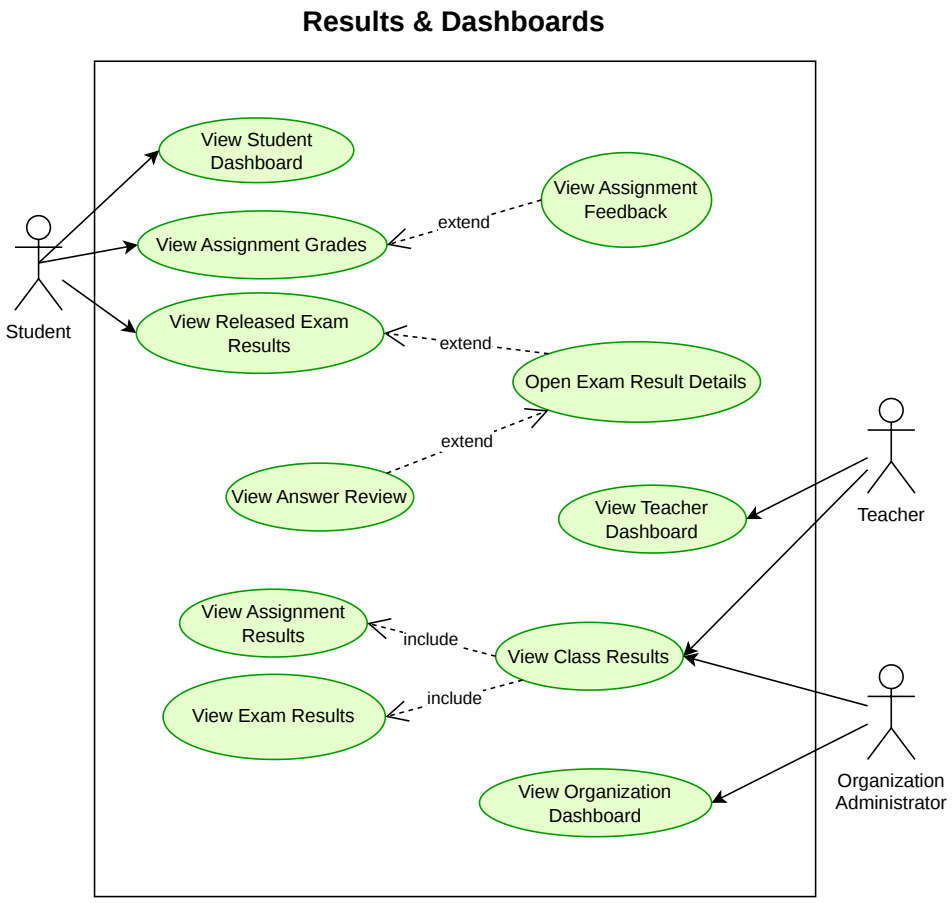

Figure 4.13. Results and Dashboards Use-Case Diagram

4.7 Activity Diagrams

The following activity diagrams were redrawn from zero based on the updated use-case set and the current Eduverse workflow descriptions. They focus on user-visible actions and important system responses, avoid unnecessary implementation detail, and do not include an activity diagram for the Project Overview because that diagram is a system summary rather than a workflow.

AD-01 Authentication and Workspace Access

Figure 4.14 models access entry, account verification, password recovery, workspace-context selection, and logout within one continuous user-facing flow.

AD-02 Organization Member Access and Public Join Link

Figure 4.15 shows both invitation-based membership onboarding and Public Join Link onboarding, including target-role selection, direct join, join-request review, and student-history attachment where applicable.

AD-03 Class Creation, Configuration, and Past Terms

Figure 4.16 describes how a class is created or updated, configured, governed, and eventually moved to Past Terms while preserving historical records.

AD-04 Course Content and Materials

Figure 4.17 captures both manager-side material operations and member-side access behavior, including search, preview, download, deletion, and AI summary availability when enabled.

AD-05 Class Communication and Notifications

Figure 4.18 combines class messaging, announcement handling, chat search, and notification review in one workflow while preserving the distinction between user-authored messages and system-generated notifications.

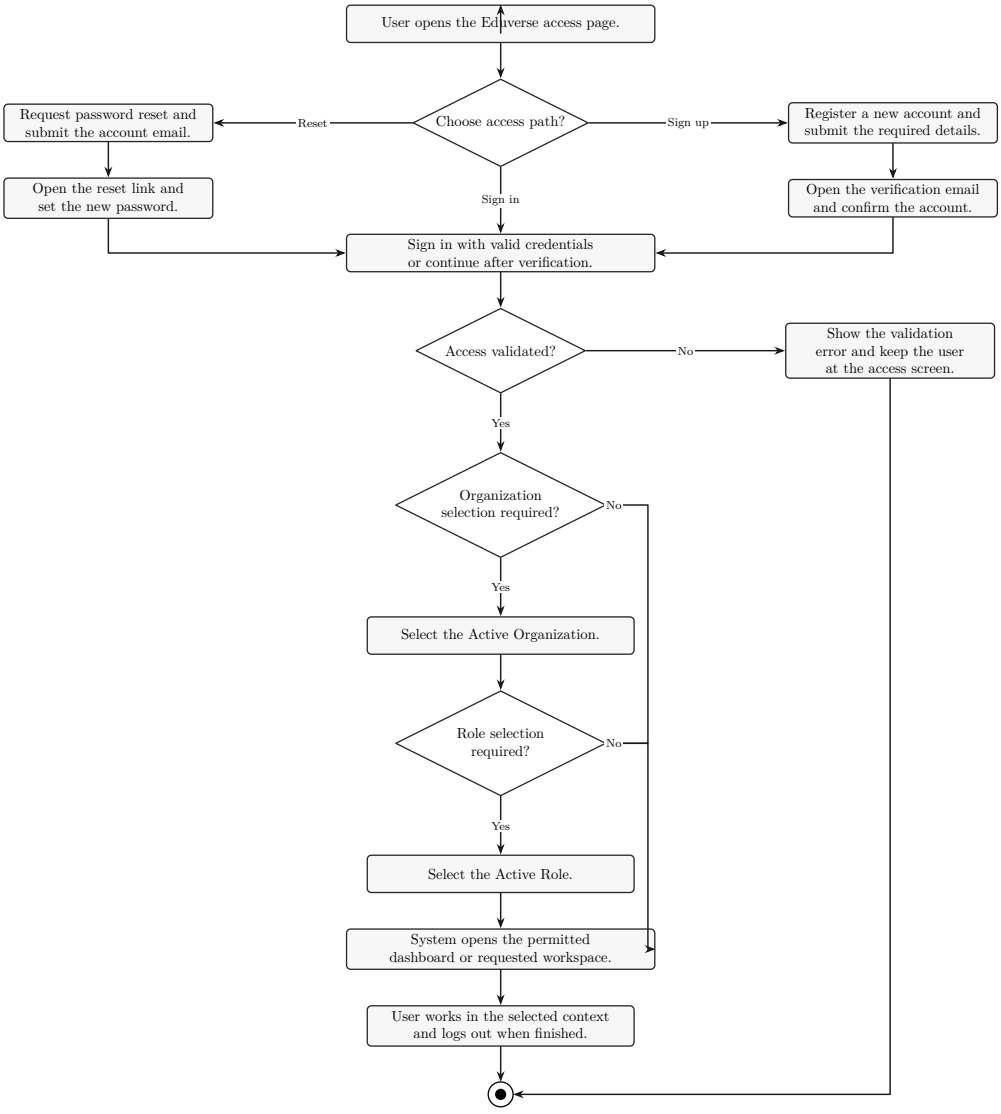

Figure 4.14. Authentication and Workspace Access Activity Diagram

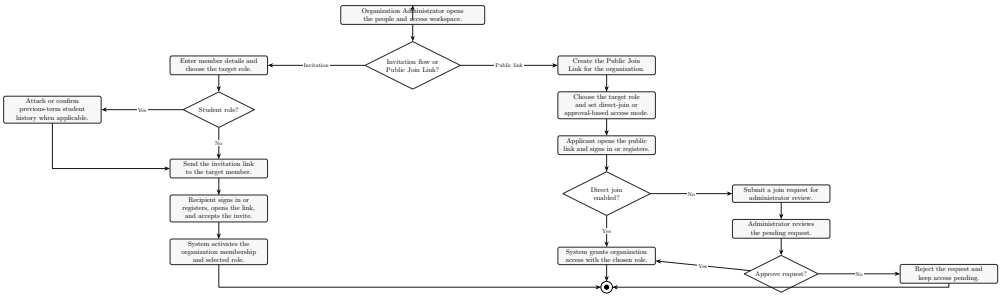

Figure 4.15. Organization Member Access and Public Join Link Activity Diagram

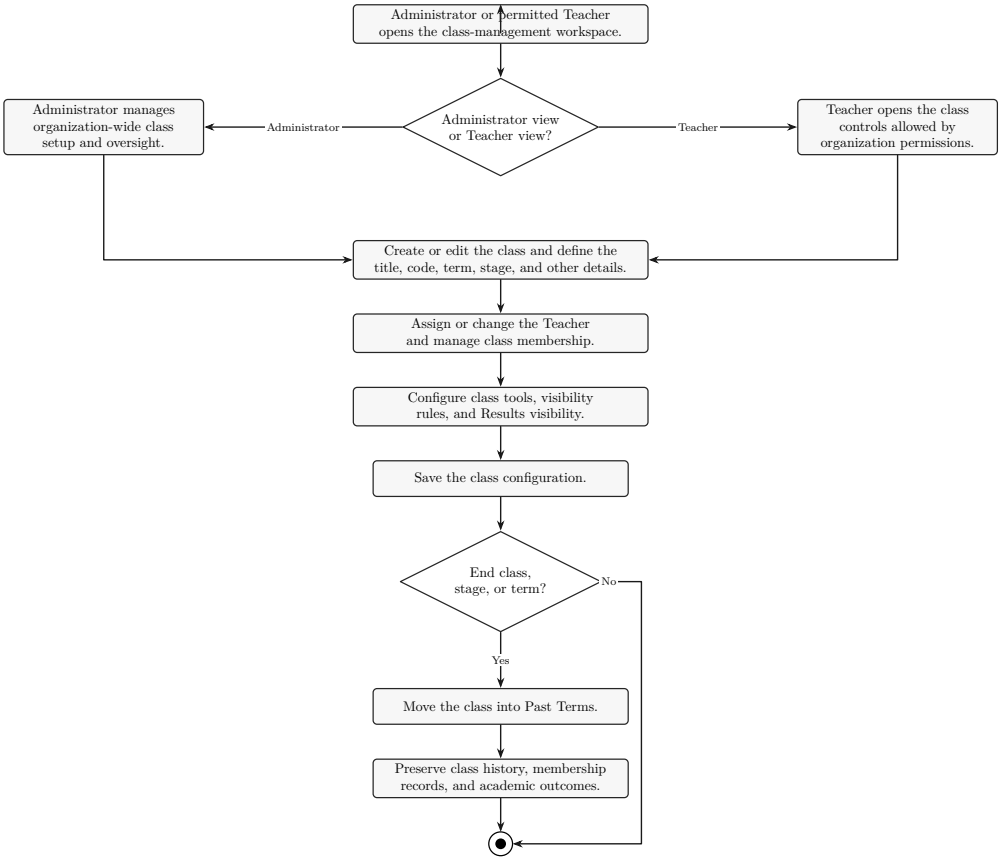

Figure 4.16. Class Creation, Configuration, and Past Terms Activity Diagram

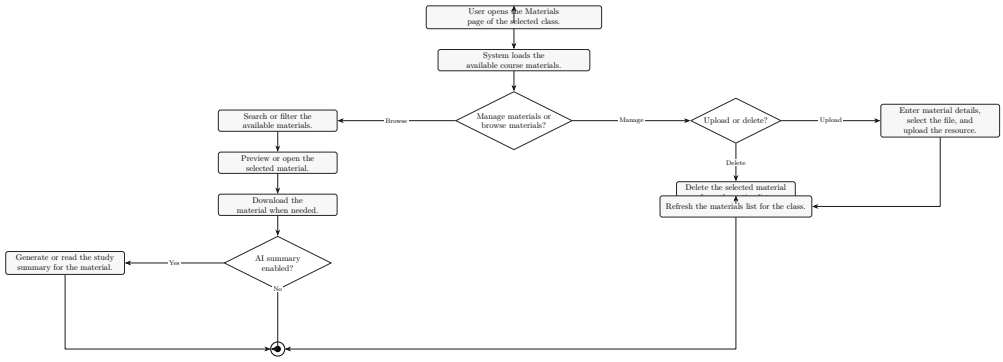

Figure 4.17. Course Content and Materials Activity Diagram

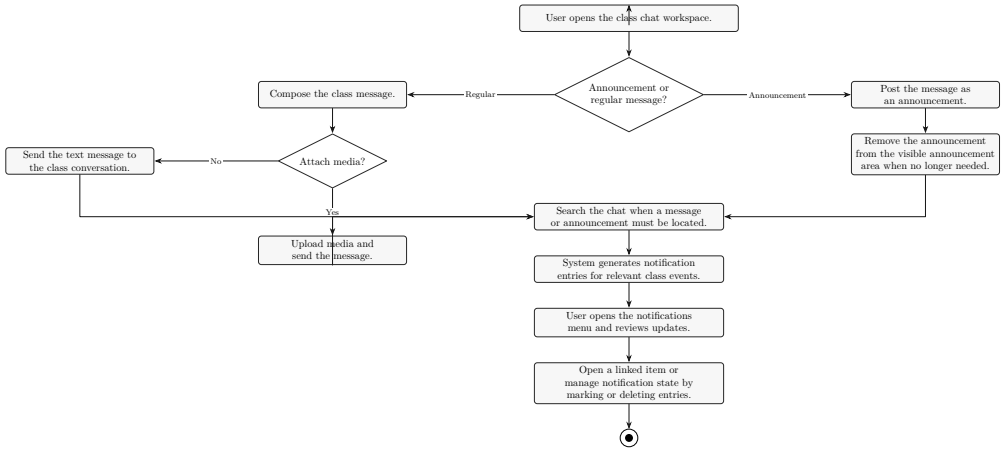

Figure 4.18. Class Communication and Notifications Activity Diagram

AD-06 Assignment Workflow

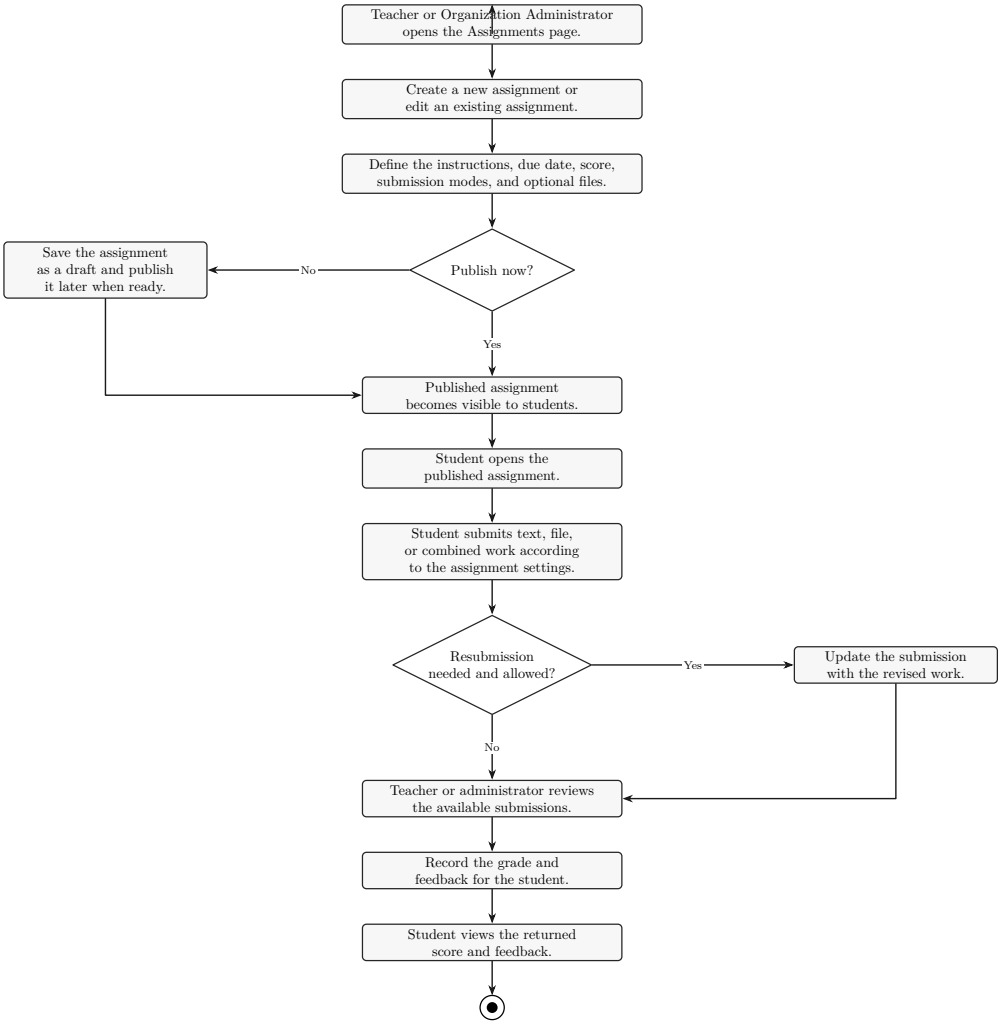

Figure 4.19. Assignment Workflow Activity Diagram

Figure 4.19 connects assignment authoring, publication, student submission, optional resubmission, grading, feedback, and student review of the returned outcome.

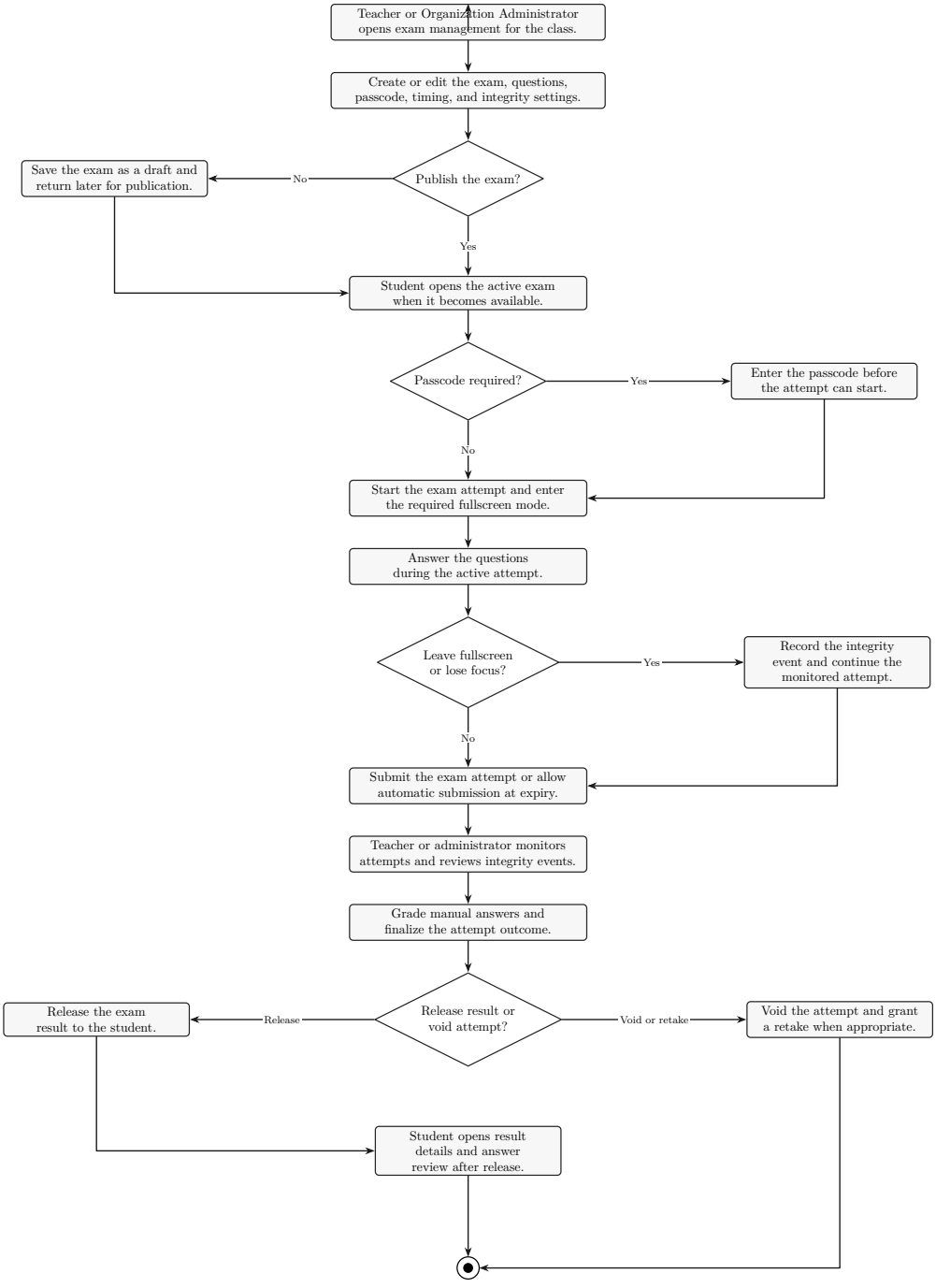

Figure 4.20. Exam Management and Integrity Workflow Activity Diagram

AD-07 Exam Management and Integrity Workflow

Figure 4.20 models exam preparation, attempt-taking, integrity-event handling, grading, release, voiding, and retake decisions in a single assessment workflow.

AD-08 Live Session and Whiteboard Workflow

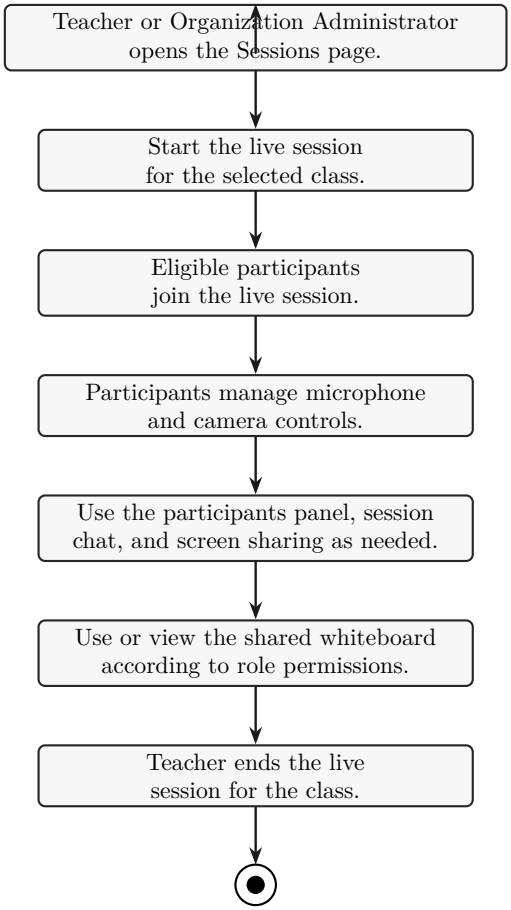

Figure 4.21. Live Session and Whiteboard Workflow Activity Diagram

Figure 4.21 represents the synchronous class session from session start through participation tools and whiteboard use to controlled session closure.

AD-09 Results and Dashboard Workflow

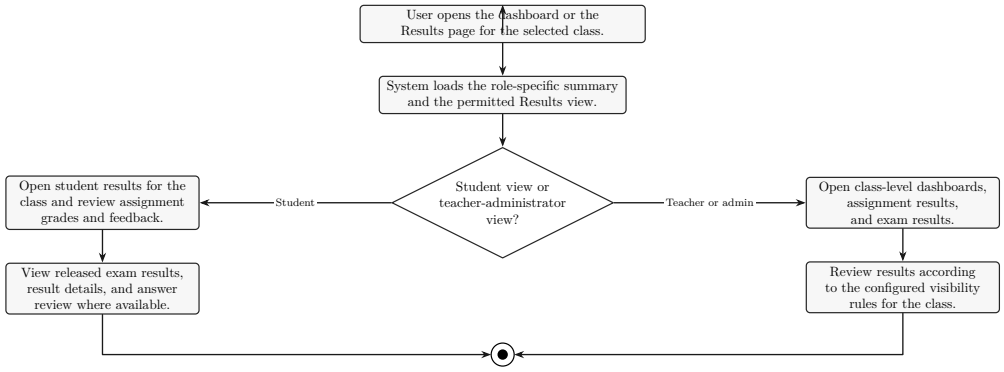

Figure 4.22. Results and Dashboard Workflow Activity Diagram

Figure 4.22 explains how the dashboard and Results area differ by actor while preserving controlled visibility for grades, feedback, released exam outcomes, and class-level summaries.

AD-10 AI Learning Support Workflow

Figure 4.23 models the main AI-enabled learning actions of Eduverse, including contextual AI questions, material summaries, assignment support, and AI-assisted drafting tasks where available.

AD-11 Integrated Coding Lab / IDE Workflow

Figure 4.24 describes entry into the coding workspace, file handling, editing, saving, running code, preview inspection, workspace reset, and terminal usage where that capability is supported.

AD-12 Profile and Help Workflow

Figure 4.25 combines profile review, theme or password settings, workspace-context switching, help browsing, and help search within one user-support workflow.

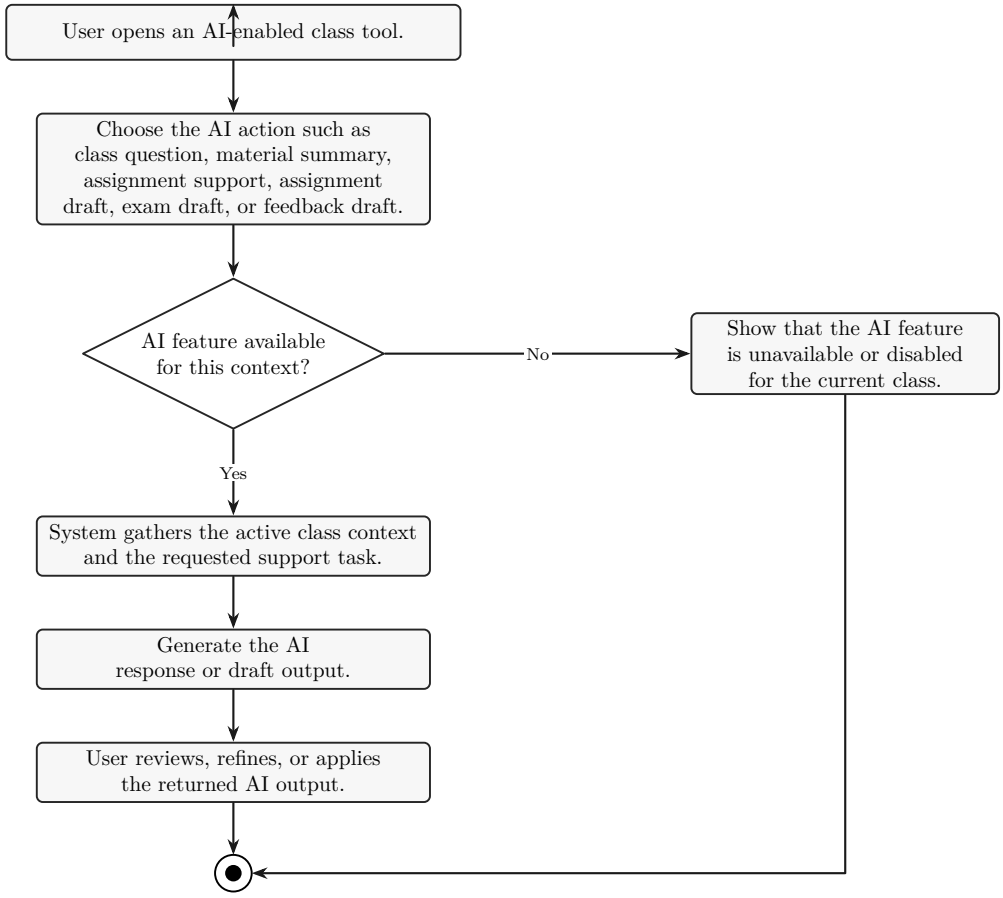

Figure 4.23. AI Learning Support Workflow Activity Diagram

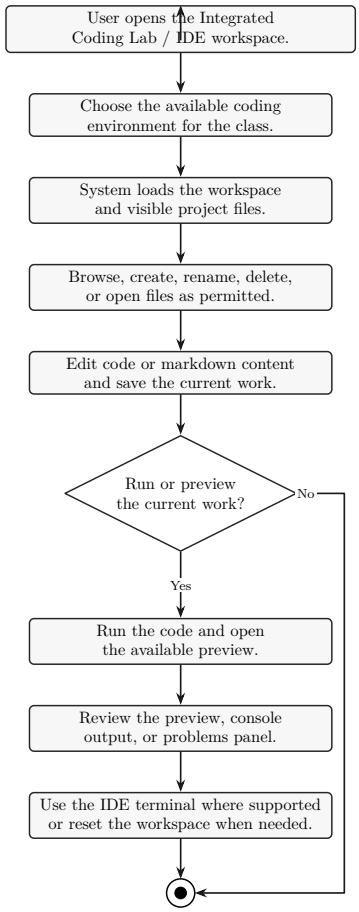

Figure 4.24. Integrated Coding Lab / IDE Workflow Activity Diagram

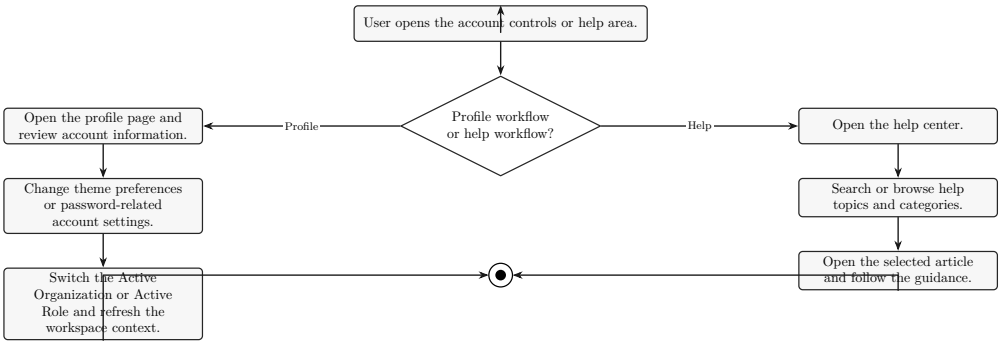

Figure 4.25. Profile and Help Workflow Activity Diagram

4.8 Chapter Summary

This chapter defined the analytical scope of Eduverse through the system overview, actor model, functional requirements, non-functional requirements, updated use-case diagrams, and redrawn activity workflows. These artifacts establish the reference basis for the following design and implementation chapters, where the structure and technical realization of the platform are explained in greater detail.

Chapter 5

System Design

Contents

5.1 Introduction

This chapter presents the system design perspective of Eduverse. In the current revision, the chapter is intentionally focused on the database design because the verified schema, regenerated ERD, and supporting database notes are sufficiently stable to document in an academic manner. The remaining design sections are retained to preserve the full chapter structure and will be expanded in later documentation passes.

5.2 Architectural Design Overview

At a high level, Eduverse combines a web-based learning environment with a mobile companion scope, a Supabase-backed PostgreSQL database, AWS S3-backed file references for uploaded academic content, and LiveKit-related session metadata for synchronous learning. A fuller component-level architectural description is reserved for a later revision, while the present section establishes the context for the detailed data design discussed in Section 5.3.

5.3 Data and Storage Design

This section documents the current verified database design of Eduverse, including the schema overview, the ERD, the physical schema tables derived from the latest analysis notes, and the main uncertainty caused by missing base-table DDL in part of the checked-in migration history.

5.3.1 Database Design Overview

Eduverse uses a Supabase-managed PostgreSQL database as the main persistent store for organizational, academic, and operational records. The schema follows a multi-organization structure in which the organization acts as the top-level tenant, while the class operates as the main instructional workspace. Within this arrangement, organization membership is modeled separately from class membership so that role selection, invitation flows, public join access, class rosters, and archived past-term history can evolve without being collapsed into one table.

The current schema persists learning content through class materials, chat-linked media records, assignments, assignment attachments, and assignment submissions. Formal assessment is represented through exams, exam questions, attempts, and answers, with released results stored directly on assignment submission records and on exam attempt or answer records rather than in separate results tables. Announcements are represented through `class_messages`, AI persistence is limited to feature settings and cached material summary or extracted-content fields, and the database does not include dedicated tables for IDE workspace state or whiteboard state.

5.3.2 Entity Relationship Diagram (ERD)

Figure 5.1 presents the current Eduverse database ERD derived from the latest checked-in schema evidence and summarizes the main organization, class, content, assessment, and operational relationships within the platform.

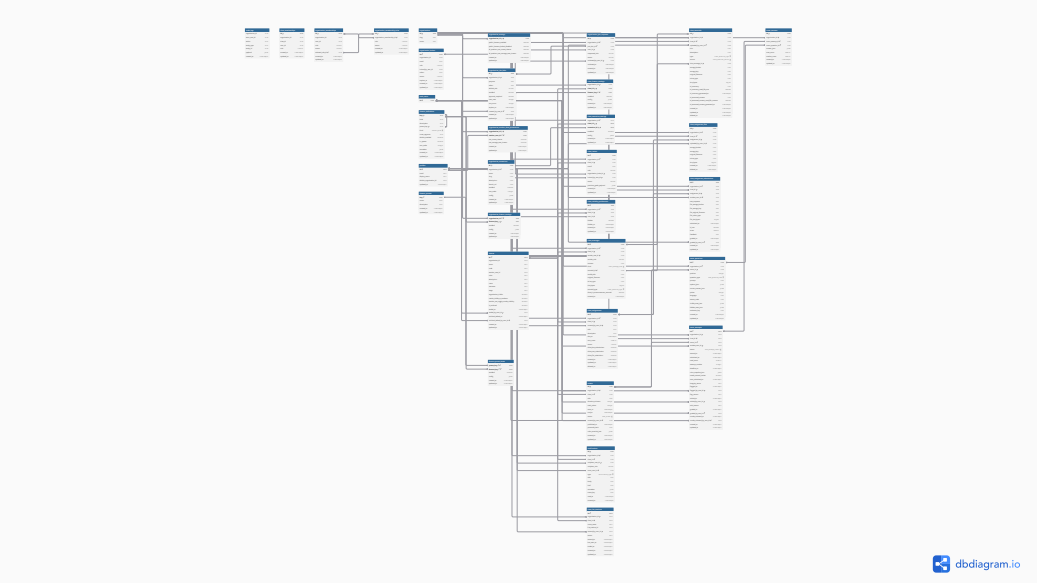

Figure 5.1. Entity Relationship Diagram of the Eduverse Database

5.3.3 Database Tables

The following schema tables present the real Eduverse database tables on normal portrait thesis pages. Column names, primary keys, foreign keys, logical relationships, enum or status fields, and essential notes are taken from `database-analysis.md`; where exact SQL types are not directly visible, conservative generic types are used.

Core Multi-Organization and User Model

Table 5.1: Schema for Table: `organizations`

Column	Type	Attributes
`id`	`uuid`	PK; original base DDL is not directly visible.
`slug`	`text`	Organization slug.
`name`	`text`	Organization display name.

Table 5.2: Schema for Table: `profiles`

Column	Type	Attributes
`id`	`uuid`	PK; logical identity mirror of `auth.users.id`.
`email`	`text`	Profile email address.
`display_name`	`text`	Visible user name.
`default_organization_id`	`uuid`	Logical relationship to `organizations.id`.
`updated_at`	`timestamp`	Profile update timestamp.

Roles, Memberships, Invitations, and Public Join Access

Table 5.3: Schema for Table: `organization_memberships`

Column	Type	Attributes
`id`	`uuid`	PK.
`organization_id`	`uuid`	Logical relationship to `organizations.id`.

Continued on the next page

Column	Type	Attributes
user_id	uuid	Logical relationship to the profile or auth identity.
role	public.app_role	Role enum.
status	public.membership_status	Membership status enum.
selected_role_id	uuid	FKtoorganization_membership_roles.id.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.

Table 5.4: Schema for Table: organization_membership_roles

Column	Type	Attributes
id	uuid	PK.
organization_membership_id	uuid	FKtoorganization_memberships.id.
role	public.app_role	Role enum.
status	public.membership_status	Membership status enum.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.

Table 5.5: Schema for Table: organization_invites

Column	Type	Attributes
id	uuid	PK; original base DDL is not directly visible.
organization_id	uuid	Logical relationship to organizations.id.
email	text	Invite target email.
role	public.app_role	Role enum.
invited_by_user_id	uuid	Logical relationship to auth.users.id.
token	text	Invite token.

Continued on the next page

Column	Type	Attributes
status	text	Membership-like status field; exact base type is not directly visible.
expires_at	timestamp	Invite expiry timestamp.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.

Table 5.6: Schema for Table: organization_settings

Column	Type	Attributes
organization_id	uuid	PK; FKtoorganizations.id.
public_features_enabled	boolean	Public-feature availability flag.
public_features_locked_disabled	boolean	Preset lock flag for disabled public features.
all_teachers_can_create_classes	boolean	Teacher class-creation permission flag.
all_teachers_can_manage_own_classes	boolean	Teacher class self-management permission flag.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.

Table 5.7: Schema for Table: organization_teacher_class_permissions

Column	Type	Attributes
organization_id	uuid	Composite PK; FKtoorganizations.id.
teacher_user_id	uuid	Composite PK; FKtoprofiles.id.
can_create_classes	boolean	Teacher override flag for class creation.
can_manage_own_classes	boolean	Teacher override flag for managing owned classes.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.

Table 5.8: Schema for Table: `organization_join_links`

Column	Type	Attributes
`id`	`uuid`	PK.
`organization_id`	`uuid`	`FKtoorganizations.id`.
`purpose`	`text`	Join-link purpose descriptor.
`token`	`text`	Join token.
`default_role`	`public.app_role`	Default role enum, constrained to teacher or student.
`enabled`	`boolean`	Link availability flag.
`approval_required`	`boolean`	Approval workflow flag.
`max_uses`	`integer`	Optional usage limit.
`use_count`	`integer`	Current usage counter.
`expires_at`	`timestamp`	Link expiry timestamp.
`created_by_user_id`	`uuid`	`FKtoauth.users.id`.
`created_at`	`timestamp`	Creation timestamp.
`updated_at`	`timestamp`	Update timestamp.

Table 5.9: Schema for Table: `organization_join_requests`

Column	Type	Attributes
`id`	`uuid`	PK.
`organization_id`	`uuid`	`FKtoorganizations.id`.
`join_link_id`	`uuid`	`FKtoorganization_join_links.id`.
`user_id`	`uuid`	`FKtoauth.users.id`.
`requested_role`	`public.app_role`	Requested role enum.
`status`	`text`	Status text; current values include `pending`, `approved`, and `rejected`.
`reviewed_by_user_id`	`uuid`	`FKtoauth.users.id`.
`reviewed_at`	`timestamp`	Review timestamp.
`created_at`	`timestamp`	Creation timestamp.
`updated_at`	`timestamp`	Update timestamp.

Table 5.10: Schema for Table: `class_invites`

Column	Type	Attributes
`id`	`uuid`	PK.
`organization_id`	`uuid`	`FKtoorganizations.id`.
`class_id`	`uuid`	`FKtoclasses.id`.
`email`	`text`	Invite target email.
`role`	`public.class_membership_role`	Class membership role enum.
`organization_invite_id`	`uuid`	`FKtoorganization_invites.id`.
`invited_by_user_id`	`uuid`	`FKtoauth.users.id`.
`status`	`public.membership_status`	Membership status enum.
`previous_grade_payload`	`json/jsonb`	Imported previous-term grade payload.
`created_at`	`timestamp`	Creation timestamp.
`updated_at`	`timestamp`	Update timestamp.

Classes and Class Visibility

Table 5.11: Schema for Table: `classes`

Column	Type	Attributes
`id`	`uuid`	PK; original base DDL is not directly visible.
`organization_id`	`uuid`	Logical relationship to `organizations.id`.
`name`	`text`	Class name.
`code`	`text`	Class code.
`teacher_user_id`	`uuid`	Logical relationship to `profiles.id`.
`color`	`text`	Class color identifier.
`description`	`text`	Class description.
`room`	`text`	Room or location field.
`semester`	`text`	Required semester text.
`stage`	`text`	Required stage text.
`organization_visible`	`boolean`	Organization-wide visibility flag.

Continued on the next page

Column	Type	Attributes
results_visible_to_students	boolean	Student results visibility flag.
teacher_can_toggle_results_visibility	boolean	Teacher permission flag for results visibility.
is_archived	boolean	Archive lifecycle flag.
ended_at	timestamp	Class end timestamp.
ended_by_user_id	uuid	FKtoauth.users.id.
archived_edited_at	timestamp	Archived-state edit timestamp.
archived_edited_by_user_id	uuid	FKtoauth.users.id.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.

Table 5.12: Schema for Table: class_memberships

Column	Type	Attributes
id	uuid	PK; original base DDL is not directly visible.
organization_id	uuid	Logical relationship to organizations.id.
class_id	uuid	Logical relationship to classes.id.
user_id	uuid	Logical relationship to the profile or auth identity.
role	text	Class-level role field; current values include teacher, student, and ta.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.

Table 5.13: Schema for Table: class_visibility_preferences

Column	Type	Attributes
id	uuid	PK.
organization_id	uuid	FKtoorganizations.id.
class_id	uuid	FKtoclasses.id.
user_id	uuid	FKtoauth.users.id.
hidden	boolean	Student hide or show flag.
hidden_at	timestamp	Hide timestamp.

Continued on the next page

Column	Type	Attributes
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.

Feature Flags and Extensions

Table 5.14: Schema for Table: feature_definitions

Column	Type	Attributes
key	text	PK.
label	text	Feature label.
description	text	Feature description.
parent_key	text	FKtofeature_definitions.key.
kind	public. feature_kind	Feature kind enum.
route_segment	text	Navigation route segment.
default_enabled	boolean	Default enablement flag.
is_system	boolean	System-managed feature flag.
sort_order	integer	Display ordering value.
metadata	json/jsonb	Feature metadata payload.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.

Table 5.15: Schema for Table: feature_presets

Column	Type	Attributes
key	text	PK.
name	text	Preset name.
description	text	Preset description.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.

Table 5.16: Schema for Table: `feature_preset_items`

Column	Type	Attributes
`preset_key`	`text`	Composite PK; `FKtofeature_presets.key`.
`feature_key`	`text`	Composite PK; `FKtofeature_definitions.key`.
`enabled`	`boolean`	Preset enablement flag.
`config`	`json/jsonb`	Preset configuration payload.
`created_at`	`timestamp`	Creation timestamp.
`updated_at`	`timestamp`	Update timestamp.

Table 5.17: Schema for Table: `organization_feature_settings`

Column	Type	Attributes
`organization_id`	`uuid`	Composite PK; `FKtoorganizations.id`.
`feature_key`	`text`	Composite PK; `FKtofeature_definitions.key`.
`enabled`	`boolean`	Organization-level feature flag.
`config`	`json/jsonb`	Configuration payload; may store lock metadata.
`created_at`	`timestamp`	Creation timestamp.
`updated_at`	`timestamp`	Update timestamp.

Table 5.18: Schema for Table: `class_feature_settings`

Column	Type	Attributes
`organization_id`	`uuid`	`FKtoorganizations.id`.
`class_id`	`uuid`	Composite PK; `FKtoclasses.id`.
`feature_key`	`text`	Composite PK; `FKtofeature_definitions.key`.
`enabled`	`boolean`	Class-level feature flag.
`config`	`json/jsonb`	Class configuration payload.
`created_at`	`timestamp`	Creation timestamp.
`updated_at`	`timestamp`	Update timestamp.

Table 5.19: Schema for Table: `organization_extensions`

Column	Type	Attributes
`id`	`uuid`	PK.
`organization_id`	`uuid`	`FKtoorganizations.id`.
`name`	`text`	Extension name.
`slug`	`text`	Extension slug.
`description`	`text`	Extension description.
`launch_url`	`text`	Extension launch URL.
`enabled`	`boolean`	Extension enablement flag.
`sort_order`	`integer`	Display ordering value.
`config`	`json/jsonb`	Extension configuration payload.
`created_at`	`timestamp`	Creation timestamp.
`updated_at`	`timestamp`	Update timestamp.

Table 5.20: Schema for Table: `class_extension_settings`

Column	Type	Attributes
`organization_id`	`uuid`	`FKtoorganizations.id`.
`class_id`	`uuid`	Composite PK; `FKtoclasses.id`.
`extension_id`	`uuid`	Composite PK; `FKtoorganization_extensions.id`.
`enabled`	`boolean`	Class-level extension enablement flag.
`config`	`json/jsonb`	Class-level extension configuration payload.
`created_at`	`timestamp`	Creation timestamp.
`updated_at`	`timestamp`	Update timestamp.

Materials, Chat, and Announcements

Table 5.21: Schema for Table: `class_materials`

Column	Type	Attributes
`id`	`uuid`	PK.
`organization_id`	`uuid`	`FKtoorganizations.id`.
`class_id`	`uuid`	`FKtoclasses.id`.
`uploaded_by_user_id`	`uuid`	`FKtoprofiles.id`.

Continued on the next page

Column	Type	Attributes
title	text	Material title.
description	text	Material description.
type	public.class_material_type	Material type enum.
source	public.class_material_source	Source enum; chat-linked media is represented here rather than in a separate media table.
chat_message_id	uuid	FKtoclass_messages.id.
storage_bucket	text	AWS S3 bucket reference.
storage_key	text	AWS S3 object key reference.
original_filename	text	Original uploaded file name.
mime_type	text	Uploaded file MIME type.
size_bytes	integer	Stored file size.
ai_summary	text	Cached AI summary text; no separate AI history table is used.
ai_summary_used_file_text	boolean	Flag indicating whether file text was used to generate the cached AI summary.
ai_summary_generated_at	timestamp	AI summary generation timestamp.
ai_extracted_content	text	Cached extracted-content field.
ai_extracted_content_used_file_content	boolean	Flag indicating whether file content was used for cached extraction.
ai_extracted_content_generated_at	timestamp	Extracted-content generation timestamp.
deleted_at	timestamp	Soft-delete timestamp.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.

Table 5.22: Schema for Table: class_messages

Column	Type	Attributes
id	uuid	PK.
organization_id	uuid	FKtoorganizations.id.
class_id	uuid	FKtoclasses.id.
sender_user_id	uuid	FKtoprofiles.id.
sender_role	text	Checked text role field; current values include student, teacher, and admin.

Continued on the next page

Column	Type	Attributes
content	text	Message body text.
kind	public.class_message_kind	Message kind enum; announcements are represented by rows where kind='announcement'.
material_id	uuid	FKtoclass_materials.id.
media_title	text	Chat-media title field.
original_filename	text	Original media file name.
mime_type	text	Media MIME type.
size_bytes	integer	Stored media size.
material_type	text	Chat-media material descriptor; exact base type is not directly visible.
show_in_announcement_carousel	boolean	Announcement carousel display flag.
created_at	timestamp	Creation timestamp.

Assignments and Submissions

Table 5.23: Schema for Table: class_assignments

Column	Type	Attributes
id	uuid	PK.
organization_id	uuid	FKtoorganizations.id.
class_id	uuid	FKtoclasses.id.
created_by_user_id	uuid	FKtoprofiles.id.
title	text	Assignment title.
description	text	Assignment description.
due_at	timestamp	Submission due timestamp.
max_score	numeric	Maximum assignment score.
status	text	Status text; current values include draft and published.
allow_late_submissions	boolean	Late-submission policy flag.
allow_text_submission	boolean	Text-submission policy flag.
allow_file_submission	boolean	File-submission policy flag.
created_at	timestamp	Creation timestamp.

Continued on the next page

Column	Type	Attributes
updated_at	timestamp	Update timestamp.
deleted_at	timestamp	Soft-delete timestamp.

Table 5.24: Schema for Table: class_assignment_files

Column	Type	Attributes
id	uuid	PK.
organization_id	uuid	FKtoorganizations.id.
class_id	uuid	FKtoclasses.id.
assignment_id	uuid	FKtoclass_assignments.id.
uploaded_by_user_id	uuid	FKtoprofiles.id.
storage_bucket	text	AWS S3 bucket reference.
storage_key	text	AWS S3 object key reference.
original_filename	text	Original attachment file name.
mime_type	text	Attachment MIME type.
size_bytes	integer	Attachment size.
created_at	timestamp	Creation timestamp.
deleted_at	timestamp	Soft-delete timestamp.

Table 5.25: Schema for Table: class_assignment_submissions

Column	Type	Attributes
id	uuid	PK.
organization_id	uuid	FKtoorganizations.id.
class_id	uuid	FKtoclasses.id.
assignment_id	uuid	FKtoclass_assignments.id.
student_user_id	uuid	FKtoprofiles.id.
text_response	text	Text submission body.
file_storage_bucket	text	AWS S3 bucket reference for submitted files.
file_storage_key	text	AWS S3 object key reference for submitted files.
file_original_filename	text	Original submitted file name.
file_mime_type	text	Submitted file MIME type.
file_size_bytes	integer	Submitted file size.

Continued on the next page

Column	Type	Attributes
submitted_at	timestamp	Submission timestamp.
is_late	boolean	Late-submission flag.
score	numeric	Stores the assignment result directly; no separate assignment-results table is used.
feedback	text	Grading feedback.
graded_at	timestamp	Grading timestamp.
graded_by_user_id	uuid	FKtoprofiles.id.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.

Exams, Attempts, Answers, and Results

Table 5.26: Schema for Table: exams

Column	Type	Attributes
id	uuid	PK.
organization_id	uuid	FKtoorganizations.id.
class_id	uuid	FKtoclasses.id.
title	text	Exam title.
duration_minutes	integer	Configured exam duration.
total_points	numeric	Total exam points.
start_at	timestamp	Exam start timestamp.
end_at	timestamp	Exam end timestamp.
status	public.exam_status	Exam status enum.
created_by_user_id	uuid	FKtoauth.users.id.
published_at	timestamp	Publish timestamp.
passcode_hash	text	Optional stored passcode hash.
rules_override_json	json/jsonb	Exam rules override payload.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.

Table 5.27: Schema for Table: exam_questions

Column	Type	Attributes
id	uuid	PK.

Continued on the next page

Column	Type	Attributes
organization_id	uuid	FKtoorganizations.id.
exam_id	uuid	FKtoexams.id.
position	integer	Question ordering value.
question_type	public.exam_question_kind	Question type enum.
prompt	text	Question prompt text.
options_json	json/jsonb	Structured option payload.
correct_answer_json	json/jsonb	Structured correct-answer payload.
points	numeric	Question score value.
language	text	Question language marker.
starter_code	text	Starter code text for coding questions.
visible_tests_json	json/jsonb	Visible test payload.
hidden_tests_json	json/jsonb	Hidden test payload.
evaluator_key	text	Evaluator identifier.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.

Table 5.28: Schema for Table: exam_attempts

Column	Type	Attributes
id	uuid	PK.
organization_id	uuid	FKtoorganizations.id.
class_id	uuid	FKtoclasses.id.
exam_id	uuid	FKtoexams.id.
student_user_id	uuid	FKtoauth.users.id.
status	public.exam_attempt_status	Attempt status enum.
started_at	timestamp	Attempt start timestamp.
submitted_at	timestamp	Attempt submission timestamp.
total_score	numeric	Stores the exam-level result directly.
attempt_number	integer	Retake or attempt counter.
deadline_at	timestamp	Effective attempt deadline.
rules_snapshot_json	json/jsonb	Applied rules snapshot payload.
needs_manual_review	boolean	Manual-review flag.
auto_submitted_at	timestamp	Automatic submission timestamp.

Continued on the next page

Column	Type	Attributes
integrity_status	text	Integrity status text; current values include clear, reported, flagged, and voided.
flagged_at	timestamp	Integrity flag timestamp.
flagged_by_user_id	uuid	FKtoauth.users.id.
flag_reason	text	Integrity flag reason.
voided_at	timestamp	Void timestamp.
voided_by_user_id	uuid	FKtoauth.users.id.
void_reason	text	Void reason.
graded_at	timestamp	Grading timestamp.
graded_by_user_id	uuid	FKtoauth.users.id.
results_released_at	timestamp	Results-release timestamp.
results_released_by_user_id	uuid	FKtoauth.users.id.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.
Exam integrity events and retake history are not stored in dedicated child tables; the current design records those events through audit_logs.		

Table 5.29: Schema for Table: exam_answers

Column	Type	Attributes
id	uuid	PK.
organization_id	uuid	FKtoorganizations.id.
exam_attempt_id	uuid	FKtoexam_attempts.id.
exam_question_id	uuid	FKtoexam_questions.id.
answer_json	json/jsonb	Stored submitted answer payload.
auto_score	numeric	Stores question-level auto-scoring directly.
teacher_score	numeric	Stores question-level teacher scoring directly.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.

Notifications and Live Sessions

Table 5.30: Schema for Table: `notifications`

Column	Type	Attributes
id	uuid	PK.
organization_id	uuid	FKtoorganizations.id.
class_id	uuid	FKtoclasses.id.
recipient_user_id	uuid	FKtoprofiles.id.
recipient_role	public.app_role	Role-scoped notification enum field.
actor_user_id	uuid	FKtoprofiles.id.
type	public.notification_type	Notification type enum.
title	text	Notification title.
body	text	Notification body text.
href	text	Navigation target.
metadata	json/jsonb	Notification metadata payload.
event_key	text	Event deduplication or tracking key.
read_at	timestamp	Read timestamp.
created_at	timestamp	Creation timestamp.

Table 5.31: Schema for Table: `class_live_sessions`

Column	Type	Attributes
id	uuid	PK.
organization_id	uuid	FKtoorganizations.id.
class_id	uuid	FKtoclasses.id.
room_name	text	Session room name.
live_session_id	text	LiveKit-related session identifier.
started_by_user_id	uuid	FKtoprofiles.id.
status	text	Session status text; current values include `pending`, `live`, and `ended`.
started_at	timestamp	Session start timestamp.
last_seen_at	timestamp	Last heartbeat timestamp.
ended_at	timestamp	Session end timestamp.
created_at	timestamp	Creation timestamp.
updated_at	timestamp	Update timestamp.
The database stores session lifecycle metadata only; no dedicated whiteboard-state table is used.		

Audit Logs and External Auth References

Table 5.32: Schema for Table: `audit_logs`

Column	Type	Attributes
`organization_id`	`uuid`	Logical relationship to `organizations.id`.
`actor_user_id`	`uuid`	Logical relationship to `auth.users.id`.
`action`	`text`	Audit action key.
`entity_type`	`text`	Polymorphic entity type discriminator.
`entity_id`	not directly visible	Polymorphic logical relationship to multiple entity tables.
`payload`	`json/jsonb`	Audit payload.
`created_at`	`timestamp`	Creation timestamp.
The checked-in migration slice does not directly expose the primary key for this base table. The same table is also used for integrity events and retake-related audit history instead of dedicated child tables.		

Table 5.33: Schema for Table: `auth.users`

Column	Type	Attributes
`id`	`uuid`	PK; external Supabase authentication user identifier.
This external table is referenced by invite, join-link, join-request, exam, and class-lifecycle user fields in the Eduverse schema.		

5.3.4 Database Design Notes and Uncertain Relationships

The original `CREATETABLE` definitions for `organizations`, `profiles`, `organization_memberships`, `organization_invites`, `classes`, `class_memberships`, and `audit_logs` are not visible in the checked-in migration history. Accordingly, the schema tables above record only the columns and relationships confirmed by visible migrations, RPC logic, and active query usage, while any non-visible link is labeled as a logical relationship rather than an enforced foreign key.

5.4 Interface and Interaction Design

Detailed interface composition, screen-level interaction flows, and supporting screenshots are intentionally deferred in the present revision. This section is therefore retained as a placeholder for the later UI or UX documentation pass without introducing screenshots at this stage.

5.5 Integration and Security Design

The broader integration and security design of Eduverse, including service boundaries, access-control flows, and environment-level protections, will be expanded in a later revision of this chapter. For the current database-focused pass, the most relevant design distinction is already captured in Section 5.3.4, where enforced foreign keys are separated from logical relationships when the visible migration history is incomplete.

Chapter 6

Software and Engineering Implementation

TODO: This chapter outline is reserved for a later documentation step.

6.1 Introduction

6.2 Web Application Implementation

6.3 Mobile Companion Implementation

6.4 Backend and Service Integration

6.5 Engineering Challenges and Resolutions

Chapter 7

System Testing and Evaluation

TODO: This chapter outline is reserved for a later documentation step.

7.1 Introduction

7.2 Testing Strategy

7.3 Functional Testing

7.4 Non-Functional Evaluation

7.5 Discussion of Results

Chapter 8

Future Work and Planned Features

TODO: This chapter outline is reserved for a later documentation step.

8.1 Introduction

8.2 Mobile Expansion

8.3 AI Agent and IDE Enhancement

8.4 Scalability and Institutional Growth

Chapter 9

Conclusion

TODO: Write the final conclusion after the main report chapters are completed.

Bibliography

[1] Hamish Coates, Richard James, and Gabrielle Baldwin. A critical examination of the effects of learning management systems on university teaching and learning. *Tertiary Education and Management*, 11(1):19–36, 2005.

[2] Declan Dagger, Alexander O'Connor, Seamus Lawless, Eddie Walsh, and Vincent P. Wade. Service-oriented e-learning platforms: From monolithic systems to flexible services. *IEEE Internet Computing*, 11(3):28–35, 2007.

[3] Phillip Dawson. Five ways to hack and cheat with bring-your-own-device electronic examinations. *British Journal of Educational Technology*, 47(4):592–600, 2016.

[4] David F. Ferraiolo, D. Richard Kuhn, and Ramaswamy Chandramouli. *Role-Based Access Control*. Artech House, 2003.

[5] D. Randy Garrison, Terry Anderson, and Walter Archer. Critical inquiry in a text-based environment: Computer conferencing in higher education. *The Internet and Higher Education*, 2(2–3):87–105, 2000.

[6] Wayne Holmes, Charles Fadel, and Maya Bialik. *Artificial Intelligence in Education: Promises and Implications for Teaching and Learning*. Center for Curriculum Redesign, 2019.

[7] LiveKit. LiveKit Documentation. `https://docs.livekit.io`, 2026. Accessed June 2, 2026.

[8] Rose Luckin, Wayne Holmes, Mark Griffiths, and Laurie B. Forcier. *Intelligence Unleashed: An Argument for AI in Education*. Pearson, 2016.

[9] Microsoft. TypeScript Handbook. `https://www.typescriptlang.org/docs/handbook/intro`, 2026. Accessed June 2, 2026.

[10] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *Computer*, 29(2):38–47, 1996.

[11] Neil Selwyn. *Education and Technology: Key Issues and Debates*. Continuum, 2011.

[12] George Siemens, Dragan Gasevic, Caroline Haythornthwaite, Shane Dawson, Simon Buckingham Shum, Rebecca Ferguson, Erik Duval, Katrien Verbert, and Ryan S. J. d. Baker. Open learning analytics: An integrated and modularized platform. Society for Learning Analytics Research, 2011.

[13] Supabase. Supabase Documentation. `https://supabase.com/docs`, 2026. Accessed June 2, 2026.

[14] Vercel. Next.js Documentation. `https://nextjs.org/docs`, 2026. Accessed June 2, 2026.

Appendix A

Appendices